

PythonProject

Version:	1.0.0
Language:	Python
Install:	pip install -r requirements.txt

Files: 16 | Lines: 2649

Project Structure

```
PythonProject/  
  .gitignore  
  AICodeDeployer.spec  
  README.md  
  core/  
    __init__.py  
    deployer.py  
    exporter.py  
    parser.py  
    project_scanner.py  
    prompt_manager.py  
  main.py  
  models/  
    __init__.py  
    project.py  
  requirements.txt  
  ui/  
    __init__.py  
    localization.py  
    main_window.py
```

Project Files

.gitignore

[text] 37 lines

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
*.egg-info/
dist/
build/
*.egg

# Virtual environment
venv/
.env/
env/
.env/

# PyInstaller
*.spec
!build.spec

# IDE
.idea/
.vscode/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db
desktop.ini

# Logs
*.log

# Distribution
output/
*.exe
```

AICodeDeployer.spec

[text] 47 lines

```
# -*- mode: python ; coding: utf-8 -*-
from PyInstaller.utils.hooks import collect_all

datas = []
binaries = []
hiddenimports = ['PIL', 'PIL.Image', 'PIL.ImageDraw', 'PIL.ImageFont', 'PIL.ImageTk', 'PyQt6', 'PyQt6.QtCore', 'PyQt6.QtGui', 'PyQt6.QtWidgets']
tmp_ret = collect_all('reportlab')
datas += tmp_ret[0]; binaries += tmp_ret[1]; hiddenimports += tmp_ret[2]
tmp_ret = collect_all('PIL')
datas += tmp_ret[0]; binaries += tmp_ret[1]; hiddenimports += tmp_ret[2]

a = Analysis(
    ['main.py'],
    pathex=['C:/Users/yanix/PycharmProjects/PythonProject'],
    binaries=binaries,
    datas=datas,
    hiddenimports=hiddenimports,
    hookspath=[],
    hooksconfig={},
    runtime_hooks=[],
```

```
excludes=['IPython', 'boto3', 'botocore', 'cv2', 'django', 'fastapi', 'flask', 'jupyter', 'jupyterlab', 'keras', 'matplotlib', 'notebook',
noarchive=False,
optimize=0,
)
pyz = PYZ(a.pure)

exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.datas,
    [],
    name='AICodeDeployer',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    upx_exclude=[],
    runtime_tmpdir=None,
    console=False,
    disable_windowed_traceback=False,
    argv_emulation=False,
    target_arch=None,
    codesign_identity=None,
    entitlements_file=None,
)
```

=====

core/__init__.py

[python] 6 lines

```
# core/__init__.py
from .parser import AIFormatParser
from .deployer import ProjectDeployer
from .exporter import ProjectExporter
from .project_scanner import ProjectScanner
from .prompt_manager import PromptManager
```

core/deployer.py

[python] 108 lines

```
=====

# core/deployer.py
import os
import shutil
from pathlib import Path
from typing import Optional
from models.project import Project
import logging

logger = logging.getLogger(__name__)

class ProjectDeployer:
    """Создание проекта на диске из модели Project"""

    def deploy(
        self,
        project: Project,
        target_dir: str,
        overwrite: bool = False
    ) -> str:
        """
        Развернуть проект.

        Args:
            project: Объект проекта
            target_dir: Корневая директория для проектов
            overwrite: Перезаписывать ли существующие файлы

        Returns:
            Путь к созданному проекту
        """
        project_dir = Path(target_dir) / project.metadata.name

        # Проверка существования
        if project_dir.exists():
            if not overwrite:
                # Добавляем суффикс
                counter = 1
                while project_dir.exists():
                    project_dir = Path(target_dir) / f"{project.metadata.name}_{counter}"
                    counter += 1
            else:
                logger.warning(f"Перезаписываю: {project_dir}")

        logger.info(f"Создаю проект: {project_dir}")

        # Создаём директорию проекта
        project_dir.mkdir(parents=True, exist_ok=True)

        # Создаём файлы
        created_files = 0
        for pf in project.files:
            file_path = project_dir / pf.relative_path

            # Создаём поддиректории
            file_path.parent.mkdir(parents=True, exist_ok=True)

            # Записываем файл
            try:
                file_path.write_text(pf.content, encoding='utf-8')
                created_files += 1
                logger.debug(f" Создан: {pf.relative_path}")
            except Exception as e:
                logger.error(f" Ошибка создания {pf.relative_path}: {e}")

        # Создаём README если его нет
        readme_exists = any(
            f.relative_path.lower() in ('readme.md', 'readme.txt', 'readme')
            for f in project.files
        )
        if not readme_exists:
            self._create_readme(project, project_dir)
```

```

        logger.info(
            f"Проект создан: {created_files}/{len(project.files)} файлов "
            f"в {project_dir}"
        )

    return str(project_dir)

def _create_readme(self, project: Project, project_dir: Path):
    """Автоматически создать README.md"""
    meta = project.metadata
    lines = [f"# {meta.name}"]

    if meta.description:
        lines.extend(["", meta.description])

    lines.extend(["", "## Структура", "``"])
    lines.append(project.get_tree())
    lines.append("``")

    if meta.install_command:
        lines.extend(["", "## Установка", f"``bash\n{meta.install_command}\n``"])

    if meta.run_command:
        lines.extend(["", "## Заныск", f"``bash\n{meta.run_command}\n``"])

    if meta.build_command:
        lines.extend(["", "## Сборка", f"``bash\n{meta.build_command}\n``"])

    lines.extend([
        "", "---",
        f"*Сгенерировано AI Code Deployer*"
    ])

    readme_path = project_dir / "README.md"
    readme_path.write_text("\n".join(lines), encoding='utf-8')

```

=====

core/exporter.py

[python] 380 lines

```

# core/exporter.py
import os
import sys
from pathlib import Path
from typing import Optional
from models.project import Project, ProjectFile, ProjectMetadata
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import mm, cm
from reportlab.platypus import (
    SimpleDocTemplate, Paragraph, Spacer, Preformatted,
    Table, TableStyle, PageBreak
)
from reportlab.lib.enums import TA_LEFT, TA_CENTER
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
import logging

logger = logging.getLogger(__name__)

def _find_system_font():
    candidates = []
    if sys.platform == "win32":
        windir = os.environ.get("WINDIR", "C:\\Windows")
        fonts_dir = os.path.join(windir, "Fonts")
        candidates = [
            os.path.join(fonts_dir, "consola.ttf"),
            os.path.join(fonts_dir, "cour.ttf"),
            os.path.join(fonts_dir, "lucon.ttf"),
            os.path.join(fonts_dir, "arial.ttf"),
            os.path.join(fonts_dir, "segoeui.ttf"),
        ]

```

```

elif sys.platform == "darwin":
    candidates = [
        "/System/Library/Fonts/Menlo.ttc",
        "/System/Library/Fonts/Courier.dfont",
        "/Library/Fonts/Arial.ttf",
    ]
else:
    candidates = [
        "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
        "/usr/share/fonts/truetype/liberation/LiberationMono-Regular.ttf",
        "/usr/share/fonts/truetype/ubuntu/UbuntuMono-R.ttf",
        "/usr/share/fonts/TTF/DejaVuSansMono.ttf",
    ]
for path in candidates:
    if os.path.exists(path):
        return path
return None

def _find_system_font_bold():
    if sys.platform == "win32":
        windir = os.environ.get("WINDIR", "C:\\Windows")
        fonts_dir = os.path.join(windir, "Fonts")
        candidates = [
            os.path.join(fonts_dir, "consolab.ttf"),
            os.path.join(fonts_dir, "courbd.ttf"),
            os.path.join(fonts_dir, "arialbd.ttf"),
            os.path.join(fonts_dir, "segoeuib.ttf"),
        ]
    elif sys.platform == "darwin":
        candidates = [
            "/System/Library/Fonts/Menlo.ttc",
            "/Library/Fonts/Arial Bold.ttf",
        ]
    else:
        candidates = [
            "/usr/share/fonts/truetype/dejavu/DejaVuSansMono-Bold.ttf",
            "/usr/share/fonts/truetype/liberation/LiberationMono-Bold.ttf",
            "/usr/share/fonts/truetype/ubuntu/UbuntuMono-B.ttf",
        ]
    for path in candidates:
        if os.path.exists(path):
            return path
    return None

def _find_sans_font():
    if sys.platform == "win32":
        windir = os.environ.get("WINDIR", "C:\\Windows")
        fonts_dir = os.path.join(windir, "Fonts")
        candidates = [
            os.path.join(fonts_dir, "segoeui.ttf"),
            os.path.join(fonts_dir, "arial.ttf"),
            os.path.join(fonts_dir, "tahoma.ttf"),
        ]
    elif sys.platform == "darwin":
        candidates = [
            "/System/Library/Fonts/Helvetica.ttc",
            "/Library/Fonts/Arial.ttf",
        ]
    else:
        candidates = [
            "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
            "/usr/share/fonts/truetype/liberation/LiberationSans-Regular.ttf",
            "/usr/share/fonts/truetype/ubuntu/Ubuntu-R.ttf",
        ]
    for path in candidates:
        if os.path.exists(path):
            return path
    return None

def _find_sans_font_bold():
    if sys.platform == "win32":
        windir = os.environ.get("WINDIR", "C:\\Windows")
        fonts_dir = os.path.join(windir, "Fonts")
        candidates = [
            os.path.join(fonts_dir, "segoeuib.ttf"),
            os.path.join(fonts_dir, "arialbd.ttf"),

```

```

        os.path.join(fonts_dir, "tahomabd.ttf"),
    ]
    elif sys.platform == "darwin":
        candidates = [
            "/Library/Fonts/Arial Bold.ttf",
        ]
    else:
        candidates = [
            "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
            "/usr/share/fonts/truetype/liberation/LiberationSans-Bold.ttf",
            "/usr/share/fonts/truetype/ubuntu/Ubuntu-B.ttf",
        ]
    for path in candidates:
        if os.path.exists(path):
            return path
    return None

```

```
class ProjectExporter:
```

```

    MONO_FONT = "Courier"
    MONO_FONT_BOLD = "Courier-Bold"
    SANS_FONT = "Helvetica"
    SANS_FONT_BOLD = "Helvetica-Bold"

    def __init__(self):
        self._register_fonts()
        self.styles = self._create_styles()

    def _register_fonts(self):
        mono = _find_system_font()
        if mono:
            try:
                pdfmetrics.registerFont(TTFont("MonoCyr", mono))
                self.MONO_FONT = "MonoCyr"
                logger.info("Mono font registered: " + mono)
            except Exception as e:
                logger.warning("Failed to register mono font: " + str(e))

        mono_bold = _find_system_font_bold()
        if mono_bold:
            try:
                pdfmetrics.registerFont(TTFont("MonoCyrBold", mono_bold))
                self.MONO_FONT_BOLD = "MonoCyrBold"
            except Exception:
                self.MONO_FONT_BOLD = self.MONO_FONT

        sans = _find_sans_font()
        if sans:
            try:
                pdfmetrics.registerFont(TTFont("SansCyr", sans))
                self.SANS_FONT = "SansCyr"
                logger.info("Sans font registered: " + sans)
            except Exception as e:
                logger.warning("Failed to register sans font: " + str(e))

        sans_bold = _find_sans_font_bold()
        if sans_bold:
            try:
                pdfmetrics.registerFont(TTFont("SansCyrBold", sans_bold))
                self.SANS_FONT_BOLD = "SansCyrBold"
            except Exception:
                self.SANS_FONT_BOLD = self.SANS_FONT

    def _create_styles(self):
        styles = getSampleStyleSheet()

        # override built-in styles to use our fonts
        styles["Title"].fontName = self.SANS_FONT_BOLD
        styles["Title"].fontSize = 20
        styles["Title"].leading = 26

        styles["Heading1"].fontName = self.SANS_FONT_BOLD
        styles["Heading1"].fontSize = 14
        styles["Heading1"].leading = 18

        styles["Normal"].fontName = self.SANS_FONT
        styles["Normal"].fontSize = 10
        styles["Normal"].leading = 13

```



```

styles.add(ParagraphStyle(
    name='CodeBlock',
    fontName=self.MONO_FONT,
    fontSize=7,
    leading=9,
    leftIndent=10,
    rightIndent=10,
    backColor=colors.Color(0.95, 0.95, 0.95),
    borderColor=colors.Color(0.8, 0.8, 0.8),
    borderWidth=0.5,
    borderPadding=5,
    spaceBefore=5,
    spaceAfter=10,
))

styles.add(ParagraphStyle(
    name='FilePath',
    fontName=self.MONO_FONT_BOLD,
    fontSize=10,
    leading=14,
    textColor=colors.Color(0.1, 0.3, 0.6),
    spaceBefore=15,
    spaceAfter=5,
))

styles.add(ParagraphStyle(
    name='MetaKey',
    fontName=self.SANS_FONT_BOLD,
    fontSize=9,
    textColor=colors.Color(0.3, 0.3, 0.3),
))

styles.add(ParagraphStyle(
    name='MetaValue',
    fontName=self.SANS_FONT,
    fontSize=9,
))

styles.add(ParagraphStyle(
    name='TreeStyle',
    fontName=self.MONO_FONT,
    fontSize=8,
    leading=11,
    leftIndent=15,
    backColor=colors.Color(0.97, 0.97, 1.0),
    borderPadding=8,
    spaceBefore=5,
    spaceAfter=10,
))

return styles

def export_pdf(self, project, output_path):
    doc = SimpleDocTemplate(
        output_path,
        pagesize=A4,
        rightMargin=1.5 * cm,
        leftMargin=1.5 * cm,
        topMargin=2 * cm,
        bottomMargin=2 * cm,
        title="Project: " + project.metadata.name,
        author="AI Code Deployer"
    )

    story = []

    # title page
    story.append(Spacer(1, 4 * cm))
    story.append(Paragraph(
        self._escape(project.metadata.name),
        self.styles['Title']
    ))

    if project.metadata.description:
        story.append(Spacer(1, 0.5 * cm))
        story.append(Paragraph(
            self._escape(project.metadata.description),
            self.styles['Normal']
        ))

```

```

story.append(Spacer(1, 1 * cm))

meta_data = self._build_meta_table(project.metadata)
if meta_data:
    table = Table(meta_data, colWidths=[4 * cm, 12 * cm])
    table.setStyle(TableStyle([
        ('FONTNAME', (0, 0), (0, -1), self.SANS_FONT_BOLD),
        ('FONTNAME', (1, 0), (1, -1), self.SANS_FONT),
        ('FONTSIZE', (0, 0), (-1, -1), 9),
        ('TEXTCOLOR', (0, 0), (0, -1), colors.Color(0.3, 0.3, 0.3)),
        ('VALIGN', (0, 0), (-1, -1), 'TOP'),
        ('BOTTOMPADDING', (0, 0), (-1, -1), 4),
        ('TOPPADDING', (0, 0), (-1, -1), 4),
    ]))
    story.append(table)

story.append(Spacer(1, 0.5 * cm))
stats_text = "Files: " + str(project.file_count) + " | Lines: " + str(project.total_lines)
story.append(Paragraph(stats_text, self.styles['Normal']))

story.append(PageBreak())

# structure
story.append(Paragraph("Project Structure", self.styles['Heading1']))
story.append(Preformatted(project.get_tree(), self.styles['TreeStyle']))
story.append(Spacer(1, 0.5 * cm))
story.append(PageBreak())

# files
story.append(Paragraph("Project Files", self.styles['Heading1']))
story.append(Spacer(1, 0.3 * cm))

for i, pf in enumerate(project.files):
    story.append(Paragraph("=" * 60, self.styles['Normal']))
    story.append(Paragraph(
        self._escape(pf.relative_path),
        self.styles['FilePath']
    ))

    if pf.description:
        story.append(Paragraph(
            "<i>" + self._escape(pf.description) + "</i>",
            self.styles['Normal']
        ))

    lang_info = "[" + pf.language + "]" if pf.language else ""
    lines_count = len(pf.content.splitlines())
    info_text = lang_info + " " + str(lines_count) + " lines"
    story.append(Paragraph(
        "<font size=7 color='gray'>" + info_text + "</font>",
        self.styles['Normal']
    ))

    code_text = pf.content
    if len(code_text) > 15000:
        code_text = code_text[:15000] + "\n\n... (truncated)"

    story.append(Preformatted(code_text, self.styles['CodeBlock']))
    story.append(Spacer(1, 0.5 * cm))

    if (i + 1) % 3 == 0 and i < len(project.files) - 1:
        story.append(PageBreak())

# ai format section
story.append(PageBreak())
story.append(Paragraph("AI Format (for reimport)", self.styles['Heading1']))
story.append(Paragraph(
    "Copy text below into AI Code Deployer to recreate this project:",
    self.styles['Normal']
))
story.append(Spacer(1, 0.3 * cm))

ai_text = project.to_ai_format()
if len(ai_text) > 30000:
    ai_text = ai_text[:30000] + "\n\n... (truncated)"

story.append(Preformatted(ai_text, self.styles['CodeBlock']))

doc.build(story)

```

```

        logger.info("PDF created: " + output_path)
        return output_path

    def export_text(self, project):
        return project.to_ai_format()

    def _build_meta_table(self, meta):
        rows = []
        fields = [
            ("Version:", meta.version),
            ("Author:", meta.author),
            ("Language:", meta.language),
            ("Framework:", meta.framework),
            ("Run:", meta.run_command),
            ("Build:", meta.build_command),
            ("Install:", meta.install_command),
        ]
        for label, value in fields:
            if value:
                rows.append([label, value])
        return rows

    def _escape(self, text):
        return (text
            .replace('&', '&amp;')
            .replace('<', '&lt;')
            .replace('>', '&gt;')
        )

```

=====

core/parser.py

[python] 259 lines

```

# core/parser.py
import re
from typing import Optional, List
from pathlib import Path
from models.project import Project, ProjectFile, ProjectMetadata
from core.prompt_manager import restore_code
import logging

logger = logging.getLogger(__name__)

class AIFormatParser:

    def parse(self, text: str) -> Project:
        text = text.strip()
        if "PROJSTART" in text:
            return self._parse_plain(text)
        if "===PROJECT_START===" in text:
            return self._parse_legacy_strict(text)
        if "[PROJECT_BEGIN]" in text:
            return self._parse_legacy_bracket(text)
        return self._parse_markdown(text)

    def _parse_plain(self, text: str) -> Project:
        project = Project()
        start = text.find("PROJSTART")
        end = text.find("PROJEND")
        if start != -1:
            text = text[start + 9:]
        if end != -1:
            text = text[:text.find("PROJEND")]
        meta_start = text.find("METASTART")
        meta_end = text.find("METAEND")
        if meta_start != -1 and meta_end != -1:
            meta_text = text[meta_start + 9:meta_end]
            project.metadata = self._parse_meta(meta_text)
        struct_start = text.find("STRUCTSTART")
        struct_end = text.find("STRUCTEND")
        if struct_start != -1 and struct_end != -1:
            raw_struct = text[struct_start + 11:struct_end].strip()
            project.structure_description = restore_code(raw_struct)

```

```

file_chunks = text.split("FILESTART")
for chunk in file_chunks[1:]:
    end_pos = chunk.find("FILEEND")
    if end_pos != -1:
        chunk = chunk[:end_pos]
    pf = self._parse_plain_file(chunk)
    if pf:
        project.files.append(pf)
logger.info("Parsed: " + project.metadata.name + " " + str(len(project.files)) + " files")
return project

def _parse_meta(self, text: str) -> ProjectMetadata:
    meta = ProjectMetadata()
    field_map = {
        "PROJECT_NAME": "name",
        "DESCRIPTION": "description",
        "VERSION": "version",
        "AUTHOR": "author",
        "LANGUAGE": "language",
        "FRAMEWORK": "framework",
        "PYTHON_VERSION": "python_version",
        "NODE_VERSION": "node_version",
        "RUN_COMMAND": "run_command",
        "BUILD_COMMAND": "build_command",
        "INSTALL_COMMAND": "install_command",
    }
    restored = restore_code(text)
    for line in restored.strip().splitlines():
        line = line.strip()
        if not line:
            continue
        colon = line.find(":")
        if colon == -1:
            continue
        key = line[:colon].strip().upper()
        value = line[colon + 1:].strip()
        if key in field_map:
            setattr(meta, field_map[key], value)
        else:
            meta.extra[key] = value
    return meta

def _parse_plain_file(self, chunk: str) -> Optional[ProjectFile]:
    lines = chunk.strip().splitlines()
    path = ""
    filetype = ""
    description = ""
    code_lines = []
    in_code = False
    for line in lines:
        stripped = line.strip()
        if stripped == "CODESTART":
            in_code = True
            continue
        if stripped == "CODEEND":
            in_code = False
            continue
        if in_code:
            code_lines.append(line)
            continue
        if stripped.startswith("FILEPATH:"):
            path = stripped[9:].strip()
        elif stripped.startswith("FILETYPE:"):
            filetype = stripped[9:].strip()
        elif stripped.startswith("FILEDESC:"):
            description = stripped[9:].strip()
    if not path:
        return None
    # восстанавливаем плейсхолдеры в пути
    path = restore_code(path)
    # если путь gitignore без точки то добавляем точку
    if path.endswith("gitignore") and not path.endswith(".gitignore"):
        parts = path.rsplit("gitignore", 1)
        path = parts[0] + ".gitignore"
    description = restore_code(description)
    filetype = restore_code(filetype)
    raw_code = "\n".join(code_lines)
    restored_code = restore_code(raw_code)
    pf = ProjectFile(relative_path=path, content=restored_code, language=filetype, description=description)

```

```

    if not pf.language:
        pf.auto_detect_language()
    return pf

def _parse_legacy_strict(self, text: str) -> Project:
    project = Project()
    match = re.search(r'===PROJECT_START===(.*)===PROJECT_END===', text, re.DOTALL)
    if match:
        text = match.group(1)
    meta_match = re.search(r'---META_START---(.*)---META_END---', text, re.DOTALL)
    if meta_match:
        project.metadata = self._parse_meta(meta_match.group(1))
    struct_match = re.search(r'---STRUCTURE_START---(.*)---STRUCTURE_END---', text, re.DOTALL)
    if struct_match:
        project.structure_description = struct_match.group(1).strip()
    file_blocks = re.findall(r'---FILE_START---(.*)---FILE_END---', text, re.DOTALL)
    for block in file_blocks:
        pf = self._parse_legacy_file(block)
        if pf:
            project.files.append(pf)
    return project

def _parse_legacy_bracket(self, text: str) -> Project:
    project = Project()
    match = re.search(r'\[PROJECT_BEGIN\](.*)\[PROJECT_FINISH\]', text, re.DOTALL)
    if match:
        text = match.group(1)
    meta_match = re.search(r'\[META_BEGIN\](.*)\[META_FINISH\]', text, re.DOTALL)
    if meta_match:
        project.metadata = self._parse_meta(meta_match.group(1))
    struct_match = re.search(r'\[STRUCT_BEGIN\](.*)\[STRUCT_FINISH\]', text, re.DOTALL)
    if struct_match:
        project.structure_description = struct_match.group(1).strip()
    file_blocks = re.findall(r'\[FILE_BEGIN\](.*)\[FILE_FINISH\]', text, re.DOTALL)
    for block in file_blocks:
        pf = self._parse_legacy_file(block)
        if pf:
            project.files.append(pf)
    return project

def _parse_legacy_file(self, block: str) -> Optional[ProjectFile]:
    lines = block.strip().splitlines()
    path = ""
    description = ""
    content_lines = []
    in_code = False
    for line in lines:
        stripped = line.strip()
        if not in_code:
            if stripped.startswith("PATH:"):
                path = stripped[5:].strip()
            elif stripped.startswith("FILEPATH:"):
                path = stripped[9:].strip()
            elif stripped.startswith("DESCRIPTION:"):
                description = stripped[12:].strip()
            elif stripped.startswith("FILEDESC:"):
                description = stripped[9:].strip()
            elif stripped.startswith("`") or stripped == "CODESTART" or stripped == "[CODE_BEGIN]":
                in_code = True
        else:
            if stripped.startswith("`") or stripped == "CODEEND" or stripped == "[CODE_FINISH]":
                in_code = False
            else:
                content_lines.append(line)
    if not path:
        return None
    pf = ProjectFile(relative_path=path, content="\n".join(content_lines), description=description)
    pf.auto_detect_language()
    return pf

def _parse_markdown(self, text: str) -> Project:
    project = Project()
    title_match = re.search(r'^#\s+(.)$', text, re.MULTILINE)
    if title_match:
        name = title_match.group(1).strip()
        name = re.sub(r'[_]', '', name)
        project.metadata.name = self._slugify(name)
    pattern = re.compile(
        r'(?!\n)(?:#\{1,6}\s+[`]*([^\n`]+\.\w+)[`]*|`{1,3}([^\n`]+\.\w+)[`]*{1,3}|(?:файл|file|create)\s*[:\n]?[`]*([^\n`]+)

```

```

        re.DOTALL | re.IGNORECASE | re.MULTILINE
    )
    for match in pattern.finditer(text):
        path = match.group(1) or match.group(2) or match.group(3)
        lang = match.group(4) or ""
        content = match.group(5)
        if path:
            path = path.strip().strip('`"'\'')
            pf = ProjectFile(relative_path=path, content=content, language=lang)
            if not pf.language:
                pf.auto_detect_language()
            project.files.append(pf)
    if not project.files:
        project.files = self._parse_simple_codeblocks(text)
    if not project.metadata.name and project.files:
        first = project.files[0].relative_path
        if '/' in first:
            project.metadata.name = Path(first).parts[0]
        else:
            project.metadata.name = Path(first).stem
    return project

def _parse_simple_codeblocks(self, text: str) -> List[ProjectFile]:
    files = []
    parts = re.split(r'```(\w*)\n', text)
    i = 0
    while i < len(parts) - 2:
        before = parts[i]
        lang = parts[i + 1]
        code_and_rest = parts[i + 2]
        code_end = code_and_rest.find('\n```')
        if code_end == -1:
            code = code_and_rest
        else:
            code = code_and_rest[:code_end]
        path = self._extract_filename(before)
        if path:
            pf = ProjectFile(relative_path=path, content=code, language=lang)
            if not pf.language:
                pf.auto_detect_language()
            files.append(pf)
        i += 3
    return files

def _extract_filename(self, text: str) -> str:
    lines = text.strip().splitlines()[-3:]
    for line in reversed(lines):
        line = line.strip()
        match = re.search(r'[*]*([a-zA-Z0-9_/\.\-]+\.[a-zA-Z0-9]+)[*]*', line)
        if match:
            candidate = match.group(1)
            if len(candidate) < 100:
                return candidate.replace('\\', '/')
    return ""

def _slugify(self, text: str) -> str:
    text = text.lower().strip()
    text = re.sub(r'^\w\s-', '', text)
    text = re.sub(r'[\s_]+', '_', text)
    return text[:50] or "project"

```

core/project_scanner.py

[python] 199 lines

```
# core/project_scanner.py
import os
import platform
from pathlib import Path
from typing import List, Optional
from models.project import Project, ProjectFile
import logging

logger = logging.getLogger(__name__)

class ProjectScanner:
    """Поиск папок проектов и сканирование существующих проектов"""

    # Типичные папки для проектов
    COMMON_PROJECT_DIRS = [
        "Projects", "projects",
        "Dev", "dev",
        "Development", "development",
        "Code", "code",
        "Workspace", "workspace",
        "src", "repos",
        "GitHub", "github",
        "GitLab", "gitlab",
    ]

    # Файлы-индикаторы того, что папка – это проект
    PROJECT_INDICATORS = [
        "package.json", "requirements.txt", "setup.py", "pyproject.toml",
        "Cargo.toml", "go.mod", "pom.xml", "build.gradle",
        "Makefile", "CMakeLists.txt", "docker-compose.yml",
        ".git", "README.md", "main.py", "app.py", "index.html",
        "manage.py", ".gitignore",
    ]

    # Игнорируемые директории
    IGNORE_DIRS = {
        '__pycache__', 'node_modules', '.git', '.svn', '.hg',
        'venv', '.venv', 'env', '.env', '.idea', '.vscode',
        'dist', 'build', '.next', '.nuxt', 'target',
        '.pytest_cache', '.mypy_cache', '__MACOSX',
        'bin', 'obj',
    }

    # Игнорируемые расширения
    IGNORE_EXTENSIONS = {
        '.pyc', '.pyo', '.class', '.o', '.obj', '.exe',
        '.dll', '.so', '.dylib', '.bin', '.dat',
        '.png', '.jpg', '.jpeg', '.gif', '.bmp', '.ico', '.svg',
        '.mp3', '.mp4', '.avi', '.mov', '.wav',
        '.zip', '.tar', '.gz', '.rar', '.7z',
        '.pdf', '.doc', '.docx', '.xls', '.xlsx',
        '.db', '.sqlite', '.sqlite3',
        '.woff', '.woff2', '.ttf', '.eot',
    }

    def find_project_directories(self) -> List[str]:
        """Найти вероятные директории с проектами"""
        home = Path.home()
        found = []

        # Проверяем стандартные пути
        for dirname in self.COMMON_PROJECT_DIRS:
            path = home / dirname
            if path.exists() and path.is_dir():
                found.append(str(path))

        # Desktop
        desktop = home / "Desktop"
        if desktop.exists():
            found.append(str(desktop))
```

```

# Documents
documents = home / "Documents"
if documents.exists():
    found.append(str(documents))

# Windows-specific
if platform.system() == "Windows":
    for drive in ['C:', 'D:', 'E:']:
        for dirname in self.COMMON_PROJECT_DIRS:
            path = Path(f"{drive}\\{dirname}")
            if path.exists():
                found.append(str(path))

# macOS-specific
if platform.system() == "Darwin":
    dev_path = home / "Developer"
    if dev_path.exists():
        found.append(str(dev_path))

# Убираем дубли, сохраняя порядок
seen = set()
unique = []
for p in found:
    normalized = os.path.normpath(p)
    if normalized not in seen:
        seen.add(normalized)
        unique.append(normalized)

return unique

def scan_project(self, project_path: str) -> Project:
    """
    Сканировать существующий проект и создать объект Project.
    """
    root = Path(project_path)

    if not root.exists():
        raise FileNotFoundError(f"Путь не найден: {project_path}")

    project = Project()
    project.metadata.name = root.name

    # Сканируем файлы
    for file_path in sorted(root.rglob('*')):
        if not file_path.is_file():
            continue

        # Проверяем, не в игнорируемой ли директории
        relative = file_path.relative_to(root)
        parts = relative.parts

        if any(part in self.IGNORE_DIRS for part in parts):
            continue

        # Проверяем расширение
        if file_path.suffix.lower() in self.IGNORE_EXTENSIONS:
            continue

        # Проверяем размер (пропускаем файлы > 500KB)
        if file_path.stat().st_size > 500 * 1024:
            logger.debug(f"Пропущен (слишком большой): {relative}")
            continue

        # Читаем содержимое
        try:
            content = file_path.read_text(encoding='utf-8')
        except (UnicodeDecodeError, PermissionError):
            try:
                content = file_path.read_text(encoding='latin-1')
            except Exception:
                logger.debug(f"Не удалось прочитать: {relative}")
                continue

        pf = ProjectFile(
            relative_path=str(relative).replace('\\', '/'),
            content=content
        )
        pf.auto_detect_language()
        project.files.append(pf)

```



```

# Попробуем определить метаданные из файлов проекта
self._detect_metadata(project)

logger.info(
    f"Просканирован проект '{project.metadata.name}': "
    f"{len(project.files)} файлов, "
    f"{project.total_lines} строк"
)

return project

def _detect_metadata(self, project: Project):
    """Определить метаданные из содержимого файлов"""
    for pf in project.files:
        fname = pf.filename.lower()

        if fname == "package.json":
            self._parse_package_json(project, pf.content)
        elif fname in ("requirements.txt", "setup.py", "pyproject.toml"):
            project.metadata.language = "Python"
            if fname == "requirements.txt":
                project.metadata.install_command = "pip install -r requirements.txt"
        elif fname == "cargo.toml":
            project.metadata.language = "Rust"
        elif fname == "go.mod":
            project.metadata.language = "Go"

def _parse_package_json(self, project: Project, content: str):
    """Извлечь инфо из package.json"""
    import json
    try:
        data = json.loads(content)
        project.metadata.name = data.get("name", project.metadata.name)
        project.metadata.description = data.get("description", "")
        project.metadata.version = data.get("version", "1.0.0")
        project.metadata.language = "JavaScript"
        project.metadata.install_command = "npm install"

        scripts = data.get("scripts", {})
        if "start" in scripts:
            project.metadata.run_command = "npm start"
        if "dev" in scripts:
            project.metadata.run_command = "npm run dev"
        if "build" in scripts:
            project.metadata.build_command = "npm run build"
    except json.JSONDecodeError:
        pass

```

=====

core/prompt_manager.py

[python] 311 lines

```

# core/prompt_manager.py

from dataclasses import dataclass
from typing import Dict

@dataclass
class PromptVariant:
    key: str
    title: str
    description: str
    icon: str
    content: str

# таблица замен: плейсхолдер -> реальный символ
REPLACEMENTS = {
    "DUNDER": "_",
    "USCORE": "-",
    "BACKTICK": "`",
    "TRIPLEBACKTICK": "```",
    "HASHSIGN": "#",
}

```

```

    "STARSIGN": "*",
    "TILDE": "~",
}

# точка в начале строки = пробел
INDENT_CHAR = "."

def restore_code(text: str) -> str:
    lines = text.split("\n")
    restored = []
    for line in lines:
        # восстанавливаем отступы: точки в начале строки -> пробелы
        indent_count = 0
        for ch in line:
            if ch == INDENT_CHAR:
                indent_count += 1
            else:
                break
        if indent_count > 0:
            line = " " * indent_count + line[indent_count:]
        # восстанавливаем плейсхолдеры
        # порядок важен: сначала длинные чтобы DUNDER не сломал USCORE
        line = line.replace("TRIPLEBACKTICK", "```")
        line = line.replace("DUNDER", "__")
        line = line.replace("BACKTICK", "`")
        line = line.replace("HASHSIGN", "#")
        line = line.replace("STARSIGN", "*")
        line = line.replace("TILDE", "~")
        line = line.replace("USCORE", "_")
        restored.append(line)
    return "\n".join(restored)

```

```

class PromptManager:

    _variants: Dict[str, PromptVariant] = {}

    @classmethod
    def _init_variants(cls):
        if cls._variants:
            return
        cls._variants["full"] = PromptVariant(key="full", title="Полный промпт", description="Максимально подробные инструкции", icon="P", content=cls._short)
        cls._variants["short"] = PromptVariant(key="short", title="Короткий промпт", description="Сжатая версия", icon="S", content=cls._short)
        cls._variants["continue"] = PromptVariant(key="continue", title="Продолжение", description="Когда ответ обрзан", icon="C", content=cls._short)
        cls._variants["refactor"] = PromptVariant(key="refactor", title="Рефакторинг", description="Переписать проект", icon="R", content=cls._short)
        cls._variants["add_feature"] = PromptVariant(key="add_feature", title="Добавить фичу", description="Добавить функционал", icon="A", content=cls._short)

    @classmethod
    def _full(cls) -> str:
        p = ""
        p += "You are a code generator. Output ONLY plain text.\n"
        p += "ABSOLUTELY NO MARKDOWN. No formatting of any kind.\n"
        p += "No triple backticks. No hash headers. No bold. No italic. No lists.\n"
        p += "\n"
        p += "CRITICAL: You must replace special characters in ALL code with safe placeholders.\n"
        p += "This is mandatory because the output will be copy-pasted through systems that break formatting.\n"
        p += "\n"
        p += "REPLACEMENT TABLE (memorize this, apply to ALL code you write):\n"
        p += "- Every underscore character must be written as USCORE\n"
        p += "- Every double underscore must be written as DUNDER (not USCOREUSCORE)\n"
        p += "- Every backtick must be written as BACKTICK\n"
        p += "- Every hash sign at start of line must be written as HASHSIGN\n"
        p += "- Every asterisk must be written as STARSIGN\n"
        p += "- Every tilde must be written as TILDE\n"
        p += "\n"
        p += "INDENTATION RULE:\n"
        p += "Do NOT use spaces or tabs for indentation in code.\n"
        p += "Instead use dots at the start of lines. Each dot = one space.\n"
        p += "Example: 4 space indent = 4 dots at start of line.\n"
        p += "\n"
        p += "EXAMPLE of how Python code must look in your output:\n"
        p += "\n"
        p += "import random\n"
        p += "\n"
        p += "class MyClass:\n"
        p += "    ...def DUNDERinitDUNDER(self, name):\n"
        p += "        .....self.name = name\n"
        p += "        .....self.USCOREprivate = 0\n"

```

```

p += "\n"
p += "....def getUSCOREValue(self):\n"
p += ".....return self.USCOREprivate\n"
p += "\n"
p += "....def DUNDERstrDUNDER(self):\n"
p += ".....return self.name\n"
p += "\n"
p += "if DUNDERnameDUNDER == DUNDERmainDUNDER:\n"
p += "....obj = MyClass(\"test\")\n"
p += "....print(obj)\n"
p += "\n"
p += "Notice: DUNDER replaces double underscore. USCORE replaces single underscore.\n"
p += "Dots at start of line replace spaces for indentation.\n"
p += "The rest of the line (after dots) uses no special indentation.\n"
p += "\n"
p += "PROJECT FORMAT:\n"
p += "\n"
p += "PROJSTART\n"
p += "METASTART\n"
p += "PROJECTUSCORENAME: exampleUSCOREproject\n"
p += "DESCRIPTION: one line description\n"
p += "VERSION: 1.0.0\n"
p += "LANGUAGE: Python\n"
p += "FRAMEWORK: none\n"
p += "RUNUSCORECOMMAND: python main.py\n"
p += "BUILDUSCORECOMMAND: none\n"
p += "INSTALLUSCORECOMMAND: pip install -r requirements.txt\n"
p += "METAEND\n"
p += "\n"
p += "STRUCTSTART\n"
p += "exampleUSCOREproject/\n"
p += "..src/\n"
p += "....DUNDERinitDUNDER.py\n"
p += "....app.py\n"
p += "..main.py\n"
p += "..requirements.txt\n"
p += "..gitignore\n"
p += "STRUCTEND\n"
p += "\n"
p += "FILESTART\n"
p += "FILEPATH: src/DUNDERinitDUNDER.py\n"
p += "FILETYPE: python\n"
p += "CODESTART\n"
p += "HASHSIGNempty init\n"
p += "CODEEND\n"
p += "FILEEND\n"
p += "\n"
p += "FILESTART\n"
p += "FILEPATH: src/app.py\n"
p += "FILETYPE: python\n"
p += "CODESTART\n"
p += "def hello():\n"
p += "....print(\"hello\")\n"
p += "CODEEND\n"
p += "FILEEND\n"
p += "\n"
p += "FILESTART\n"
p += "FILEPATH: main.py\n"
p += "FILETYPE: python\n"
p += "CODESTART\n"
p += "from src.app import hello\n"
p += "hello()\n"
p += "CODEEND\n"
p += "FILEEND\n"
p += "\n"
p += "PROJEND\n"
p += "\n"
p += "RULES:\n"
p += "1. EVERY underscore in code and filenames must be USCORE or DUNDER. No raw underscores anywhere.\n"
p += "2. EVERY indent must be dots. No spaces or tabs at line start. 4 dots = 4 spaces.\n"
p += "3. PROJSTART PROJEND METASTART METAEND STRUCTSTART STRUCTEND FILESTART FILEEND CODESTART CODEEND are markers. Each on its own\n"
p += "4. Every file from STRUCT must have its own FILESTART block.\n"
p += "5. FILEPATH must match path in STRUCT exactly.\n"
p += "6. Code must be complete and working. No placeholders. No TODO.\n"
p += "7. No text before PROJSTART or after PROJEND.\n"
p += "8. No markdown formatting anywhere.\n"
p += "9. Apply replacements even in strings, comments, everywhere.\n"
p += "10. The structure tree uses dots for indentation too, 2 dots per level.\n"
p += "11. For .gitignore file write gitignore without dot in STRUCT and FILEPATH. The dot will be added automatically.\n"

```

```

        return p

@classmethod
def _short(cls) -> str:
    p = ""
    p += "Output ONLY plain text. No markdown. No formatting.\n"
    p += "Replace underscores with USCORE, double underscores with DUNDER.\n"
    p += "Use dots for indentation (1 dot = 1 space).\n"
    p += "Replace hash at line start with HASHSIGN.\n"
    p += "\n"
    p += "PROJSTART\n"
    p += "METASTART\n"
    p += "PROJECTUSCORENAME: name\n"
    p += "DESCRIPTION: description\n"
    p += "LANGUAGE: language\n"
    p += "RUNUSCORECOMMAND: run command\n"
    p += "INSTALLUSCORECOMMAND: install command\n"
    p += "METAEND\n"
    p += "STRUCTSTART\n"
    p += "file tree with 2 dot indentation\n"
    p += "STRUCTEND\n"
    p += "For EACH file:\n"
    p += "FILESTART\n"
    p += "FILEPATH: path\n"
    p += "FILETYPE: language\n"
    p += "CODESTART\n"
    p += "complete code with dot indentation and USCORE/DUNDER replacements\n"
    p += "CODEEND\n"
    p += "FILEEND\n"
    p += "PROJEND\n"
    return p

@classmethod
def _cont(cls) -> str:
    p = ""
    p += "Previous response was cut off. Write the REMAINING files only.\n"
    p += "Same rules: dots for indent, USCORE for underscore, DUNDER for double underscore.\n"
    p += "No markdown. Start with FILESTART. End with PROJEND.\n"
    p += "\n"
    p += "FILESTART\n"
    p += "FILEPATH: path\n"
    p += "FILETYPE: language\n"
    p += "CODESTART\n"
    p += "complete code\n"
    p += "CODEEND\n"
    p += "FILEEND\n"
    p += "\n"
    p += "PROJEND\n"
    return p

@classmethod
def _refactor(cls) -> str:
    p = ""
    p += "Below is a project. Refactor it. Output in same format PROJSTART to PROJEND.\n"
    p += "Same rules: dots for indent, USCORE/DUNDER for underscores. No markdown.\n"
    p += "\n"
    p += "Source project:\n"
    p += "\n"
    return p

@classmethod
def _add_feature(cls) -> str:
    p = ""
    p += "Below is a project. Add new functionality. Output ENTIRE project PROJSTART to PROJEND.\n"
    p += "Same rules: dots for indent, USCORE/DUNDER for underscores. No markdown.\n"
    p += "\n"
    p += "Source project:\n"
    p += "\n"
    return p

@classmethod
def get_prompt(cls, variant: str = "full") -> str:
    cls._init_variants()
    v = cls._variants.get(variant)
    if v is None:
        raise KeyError("Unknown variant: " + variant)
    return v.content

@classmethod

```

```

def get_variant(cls, key: str) -> PromptVariant:
    cls._init_variants()
    v = cls._variants.get(key)
    if v is None:
        raise KeyError("Unknown variant: " + key)
    return v

@classmethod
def get_all_variants(cls) -> Dict[str, PromptVariant]:
    cls._init_variants()
    return dict(cls._variants)

@classmethod
def get_variant_keys(cls) -> list:
    cls._init_variants()
    return list(cls._variants.keys())

@classmethod
def build_project_request(cls, description, variant="full", language="", framework="", extra_requirements=""):
    prompt = cls.get_prompt(variant)
    parts = [prompt, "", "TASK:", description]
    if language:
        parts.append("Preferred language: " + language)
    if framework:
        parts.append("Preferred framework: " + framework)
    if extra_requirements:
        parts.append("Extra requirements: " + extra_requirements)
    return "\n".join(parts)

@classmethod
def build_continue_request(cls, missing_files=None):
    prompt = cls.get_prompt("continue")
    if missing_files:
        prompt += "\nFiles still needed:\n"
        for f in missing_files:
            prompt += f + "\n"
    return prompt

@classmethod
def build_refactor_request(cls, project_ai_text, instructions=""):
    prompt = cls.get_prompt("refactor")
    prompt += project_ai_text
    if instructions:
        prompt += "\n\nWhat to improve:\n" + instructions
    return prompt

@classmethod
def build_add_feature_request(cls, project_ai_text, feature_description):
    prompt = cls.get_prompt("add_feature")
    prompt += project_ai_text
    prompt += "\n\nNew feature:\n" + feature_description
    return prompt

```

=====

main.py

[python] 39 lines

```

import sys
import logging
from PyQt6.QtWidgets import QApplication
from PyQt6.QtGui import QFont
from ui.main_window import MainWindow

def setup_logging():
    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
        handlers=[
            logging.FileHandler('deployer.log', encoding='utf-8'),
            logging.StreamHandler()
        ]
    )

```

```
def main():
    setup_logging()
    logger = logging.getLogger(__name__)
    logger.info("Занык AI Code Deployer")

    app = QApplication(sys.argv)
    app.setApplicationName("AI Code Deployer")
    app.setStyle("Fusion")

    # Шрифт по умолчанию
    font = QFont("Segoe UI", 10)
    app.setFont(font)

    window = MainWindow()
    window.show()

    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

models/__init__.py

[python] 2 lines

```
# models/__init__.py
from .project import Project, ProjectFile, ProjectMetadata
```

models/project.py

[python] 173 lines

```
# models/project.py
from dataclasses import dataclass, field
from typing import List, Dict
from pathlib import Path

@dataclass
class ProjectFile:
    relative_path: str
    content: str
    language: str = ""
    description: str = ""

    @property
    def filename(self) -> str:
        return Path(self.relative_path).name

    @property
    def directory(self) -> str:
        parent = str(Path(self.relative_path).parent)
        return "" if parent == "." else parent

    @property
    def extension(self) -> str:
        return Path(self.relative_path).suffix.lstrip(".")

    def auto_detect_language(self):
        ext_map = {
            "py": "python", "js": "javascript", "ts": "typescript",
            "jsx": "jsx", "tsx": "tsx", "html": "html", "htm": "html",
            "css": "css", "scss": "scss", "sass": "sass",
            "java": "java", "kt": "kotlin", "swift": "swift",
            "c": "c", "cpp": "cpp", "h": "c", "hpp": "cpp",
            "cs": "csharp", "go": "go", "rs": "rust",
            "rb": "ruby", "php": "php", "lua": "lua",
            "sql": "sql", "sh": "bash", "bat": "batch",
            "json": "json", "xml": "xml", "yaml": "yaml",
            "yml": "yaml", "toml": "toml", "ini": "ini",
            "md": "markdown", "txt": "text",
            "dockerfile": "dockerfile", "gitignore": "text",
        }
        ext = self.extension.lower()
        fname = self.filename.lower()
        if fname == "dockerfile":
            self.language = "dockerfile"
        elif fname == ".gitignore":
            self.language = "text"
        elif ext in ext_map:
            self.language = ext_map[ext]
        else:
            self.language = "text"

@dataclass
class ProjectMetadata:
    name: str = "unnamed_project"
    description: str = ""
    version: str = "1.0.0"
    author: str = ""
    language: str = ""
```

```

framework: str = ""
python_version: str = ""
node_version: str = ""
run_command: str = ""
build_command: str = ""
install_command: str = ""
extra: Dict[str, str] = field(default_factory=dict)

```

```

def encode_for_ai(text: str) -> str:
    # обратная операция: код -> плейсхолдеры для AI формата
    text = text.replace("__", "DUNDER")
    text = text.replace("_", "USCORE")
    text = text.replace("`", "BACKTICK")
    text = text.replace("~", "TILDE")
    text = text.replace("*", "STARSIGN")
    lines = text.split("\n")
    encoded = []
    for line in lines:
        indent_count = 0
        for ch in line:
            if ch == " ":
                indent_count += 1
            else:
                break
        if indent_count > 0:
            line = "." * indent_count + line[indent_count:]
        # hash at start of line
        stripped_after_dots = line.lstrip(".")
        if stripped_after_dots.startswith("#"):
            dot_part = line[:len(line) - len(stripped_after_dots)]
            line = dot_part + "HASHSIGN" + stripped_after_dots[1:]
        encoded.append(line)
    return "\n".join(encoded)

```

```

@dataclass
class Project:
    metadata: ProjectMetadata = field(default_factory=ProjectMetadata)
    files: List[ProjectFile] = field(default_factory=list)
    structure_description: str = ""

```

```

@property
def file_count(self) -> int:
    return len(self.files)

```

```

@property
def total_lines(self) -> int:
    return sum(len(f.content.splitlines()) for f in self.files)

```

```

def get_tree(self) -> str:
    if not self.files:
        return "(empty)"
    lines = [self.metadata.name + "/"]
    paths = sorted(f.relative_path for f in self.files)
    seen_dirs = set()
    for p in paths:
        parts = Path(p).parts
        for i in range(len(parts) - 1):
            dir_path = "/".join(parts[:i + 1])
            if dir_path not in seen_dirs:
                seen_dirs.add(dir_path)
                indent = " " * (i + 1)
                lines.append(indent + parts[i] + "/")
        indent = " " * len(parts)
        lines.append(indent + parts[-1])
    return "\n".join(lines)

```

```

def to_ai_format(self) -> str:
    out = ""
    out += "PROJSTART\n"
    out += "METASTART\n"
    out += "PROJECT_NAME: " + self.metadata.name + "\n"
    if self.metadata.description:
        out += "DESCRIPTION: " + self.metadata.description + "\n"
    if self.metadata.version:
        out += "VERSION: " + self.metadata.version + "\n"
    if self.metadata.author:
        out += "AUTHOR: " + self.metadata.author + "\n"

```



```

if self.metadata.language:
    out += "LANGUAGE: " + self.metadata.language + "\n"
if self.metadata.framework:
    out += "FRAMEWORK: " + self.metadata.framework + "\n"
if self.metadata.run_command:
    out += "RUN_COMMAND: " + self.metadata.run_command + "\n"
if self.metadata.build_command:
    out += "BUILD_COMMAND: " + self.metadata.build_command + "\n"
if self.metadata.install_command:
    out += "INSTALL_COMMAND: " + self.metadata.install_command + "\n"
out += "METAEND\n"
out += "\n"
out += "STRUCTSTART\n"
tree = self.get_tree()
out += encode_for_ai(tree) + "\n"
out += "STRUCTEND\n"
for f in self.files:
    out += "\n"
    out += "FILESTART\n"
    filepath = f.relative_path
    if filepath.endswith(".gitignore"):
        filepath = filepath.replace(".gitignore", "gitignore")
    out += "FILEPATH: " + encode_for_ai(filepath) + "\n"
    if f.description:
        out += "FILEDESC: " + f.description + "\n"
    lang = f.language or f.extension or "text"
    out += "FILETYPE: " + lang + "\n"
    out += "CODESTART\n"
    out += encode_for_ai(f.content) + "\n"
    out += "CODEEND\n"
    out += "FILEEND\n"
out += "\n"
out += "PROJEND\n"
return out

```

=====

README.md

[markdown] 0 lines

requirements.txt

[text] 4 lines

```
PyQt6>=6.5.0
reportlab>=4.0
Pygments>=2.16.0
pyinstaller>=6.0
```

ui/__init__.py

[python] 3 lines

```
# ui/__init__.py
from .main_window import MainWindow
from .localization import Localization
```

ui/localization.py

[python] 271 lines

```
# ui/localization.py

TRANSLATIONS = {
    "ru": {
        # Window
        "app_title": "AI Code Deployer",

        # Directory panel
        "working_directory": "Рабочая директория",
        "browse": "Обзор",
        "refresh": "Обновить",

        # Tabs
        "tab_import": "Импорт",
        "tab_export": "Экспорт",
        "tab_help": "Помощь",

        # Import tab
        "copy_prompt_for_ai": "Скопировать промпт для ИИ:",
        "paste_ai_response": "Вставьте ответ ИИ:",
        "chars_count": "{} символов",
        "placeholder_input": (
            "Вставьте ответ ИИ сюда...\n\n"
            "Поддерживаемые форматы:\n"
            "  PROJSTART...PROJEND (из промпта)\n"
            "  Markdown блоки кода\n\n"
            "Совет: нажмите зелёную кнопку выше, чтобы скопировать промпт."
        ),
        "analyze": "Анализировать",
        "create_project": "Создать проект",
        "overwrite_existing": "Перезаписать существующий",
        "clear": "Очистить",
        "analysis_result": "Результат анализа:",

        # Export tab
        "select_project_to_export": "Выберите проект для экспорта",
        "find_projects": "Найти проекты",
        "browse_folder": "Выбрать папку",
        "export_pdf": "Экспорт PDF",
        "copy_ai_format": "Копировать AI формат",
        "export_preview": "Предпросмотр экспорта:",

        # Help tab
        "help_title": "Как использовать AI Code Deployer",
        "help_content": (
```

```

        "<h2>Быстрый старт</h2>"
        "<ol>"
        "<li><b>Скопируйте промпт</b> – нажмите зелёную кнопку на вкладке Импорт</li>"
        "<li><b>Отправьте промпт ИИ</b> – вместе с описанием вашего проекта</li>"
        "<li><b>Вставьте ответ</b> – в текстовое поле на вкладке Импорт</li>"
        "<li><b>Анализируйте</b> – нажмите кнопку Анализировать</li>"
        "<li><b>Создайте проект</b> – нажмите Создать проект</li>"
        "</ol>"
        "<h2>Экспорт</h2>"
        "<p>Вкладка Экспорт позволяет сканировать существующие проекты и экспортировать их в PDF или AI формат.</p>"
        "<h2>Горячие клавиши</h2>"
        "<ul>"
        "<li><b>Ctrl+V</b> – Вставить</li>"
        "<li><b>Ctrl+Enter</b> – Анализировать</li>"
        "<li><b>Ctrl+D</b> – Создать проект</li>"
        "<li><b>Ctrl+E</b> – Экспорт PDF</li>"
        "<li><b>Ctrl+T</b> – Сменить тему</li>"
        "</ul>"
    ),

    # Theme button
    "theme_dark": "Тема: Тёмная",
    "theme_light": "Тема: Светлая",

    # Language button
    "lang_label": "Язык: Русский",

    # Status
    "ready": "Готово",
    "analyzing": "Анализ...",
    "deploying": "Создание проекта...",
    "exporting_pdf": "Экспорт PDF...",
    "scanning": "Сканирование...",

    # Messages
    "no_text_to_analyze": "Нет текста для анализа",
    "paste_text_first": "Сначала вставьте текст ответа ИИ.",
    "analysis_complete": "Анализ завершён",
    "project_name": "Проект: {}",
    "files_count": "Файлов: {}",
    "total_lines": "Строк: {}",
    "file_list": "Файлы:",
    "analysis_error": "Ошибка анализа",
    "no_project_analyzed": "Нет проанализированного проекта",
    "analyze_first": "Сначала проанализируйте ответ ИИ.",
    "no_directory_selected": "Директория не выбрана",
    "select_working_directory": "Выберите рабочую директорию.",
    "project_created": "Проект создан",
    "project_created_at": "Проект создан в:\n{}",
    "deploy_error": "Ошибка создания проекта",
    "prompt_copied": "Промпт '{}' скопирован в буфер обмена!",
    "text_cleared": "Текст очищен",
    "select_project_folder": "Выбрать папку проекта",
    "no_project_selected": "Проект не выбран",
    "select_project_first": "Выберите или найдите проект для экспорта.",
    "project_not_found": "Проект не найден",
    "path_not_exists": "Путь не существует: {}",
    "save_pdf": "Сохранить PDF",
    "pdf_exported": "PDF экспортирован",
    "pdf_saved_at": "PDF сохранён:\n{}",
    "pdf_export_error": "Ошибка экспорта PDF",
    "ai_format_copied": "AI формат скопирован в буфер обмена!",
    "scan_error": "Ошибка сканирования",
    "found_projects": "Найдено проектов: {}",
    "select_directory": "Выбрать директорию",

    # Prompt variants
    "prompt_full": "Полный промпт",
    "prompt_short": "Короткий промпт",
    "prompt_continue": "Продолжение",
    "prompt_refactor": "Рефакторинг",
    "prompt_add_feature": "Добавить фичу",
},

"en": {
    # Window
    "app_title": "AI Code Deployer",

    # Directory panel

```

```

"working_directory": "Working Directory",
"browse": "Browse",
"refresh": "Refresh",

# Tabs
"tab_import": "Import",
"tab_export": "Export",
"tab_help": "Help",

# Import tab
"copy_prompt_for_ai": "Copy prompt for AI:",
"paste_ai_response": "Paste AI response:",
"chars_count": "{} chars",
"placeholder_input": (
    "Paste AI response here...\n\n"
    "Supported formats:\n"
    "  PROJSTART...PROJEND (from prompt)\n"
    "  Markdown code blocks\n\n"
    "Tip: click green button above to copy prompt first."
),
"analyze": "Analyze",
"create_project": "Create Project",
"overwrite_existing": "Overwrite existing",
"clear": "Clear",
"analysis_result": "Analysis Result:",

# Export tab
"select_project_to_export": "Select project to export",
"find_projects": "Find Projects",
"browse_folder": "Browse Folder",
"export_pdf": "Export PDF",
"copy_ai_format": "Copy AI Format",
"export_preview": "Export Preview:",

# Help tab
"help_title": "How to use AI Code Deployer",
"help_content": (
    "<h2>Quick Start</h2>"
    "<ol>"
    "<li><b>Copy prompt</b> – click the green button on the Import tab</li>"
    "<li><b>Send prompt to AI</b> – along with your project description</li>"
    "<li><b>Paste response</b> – into the text field on the Import tab</li>"
    "<li><b>Analyze</b> – click the Analyze button</li>"
    "<li><b>Create project</b> – click Create Project</li>"
    "</ol>"
    "<h2>Export</h2>"
    "<p>The Export tab allows you to scan existing projects and export them to PDF or AI format.</p>"
    "<h2>Keyboard Shortcuts</h2>"
    "<ul>"
    "<li><b>Ctrl+V</b> – Paste</li>"
    "<li><b>Ctrl+Enter</b> – Analyze</li>"
    "<li><b>Ctrl+D</b> – Deploy project</li>"
    "<li><b>Ctrl+E</b> – Export PDF</li>"
    "<li><b>Ctrl+T</b> – Toggle theme</li>"
    "</ul>"
),

# Theme button
"theme_dark": "Theme: Dark",
"theme_light": "Theme: Light",

# Language button
"lang_label": "Lang: English",

# Status
"ready": "Ready",
"analyzing": "Analyzing...",
"deploying": "Deploying...",
"exporting_pdf": "Exporting PDF...",
"scanning": "Scanning...",

# Messages
"no_text_to_analyze": "No text to analyze",
"paste_text_first": "Paste AI response text first.",
"analysis_complete": "Analysis complete",
"project_name": "Project: {}",
"files_count": "Files: {}",
"total_lines": "Lines: {}",
"file_list": "Files:",

```

```

        "analysis_error": "Analysis error",
        "no_project_analyzed": "No project analyzed",
        "analyze_first": "Analyze AI response first.",
        "no_directory_selected": "No directory selected",
        "select_working_directory": "Select a working directory.",
        "project_created": "Project created",
        "project_created_at": "Project created at:\n{}",
        "deploy_error": "Deploy error",
        "prompt_copied": "Prompt '{}' copied to clipboard!",
        "text_cleared": "Text cleared",
        "select_project_folder": "Select project folder",
        "no_project_selected": "No project selected",
        "select_project_first": "Select or find a project to export.",
        "project_not_found": "Project not found",
        "path_not_exists": "Path does not exist: {}",
        "save_pdf": "Save PDF",
        "pdf_exported": "PDF exported",
        "pdf_saved_at": "PDF saved at:\n{}",
        "pdf_export_error": "PDF export error",
        "ai_format_copied": "AI format copied to clipboard!",
        "scan_error": "Scan error",
        "found_projects": "Found projects: {}",
        "select_directory": "Select directory",

        # Prompt variants
        "prompt_full": "Full Prompt",
        "prompt_short": "Short Prompt",
        "prompt_continue": "Continue",
        "prompt_refactor": "Refactor",
        "prompt_add_feature": "Add Feature",
    }
}

```

```

class Localization:
    _instance = None

    def __init__(self):
        self._lang = "ru"

    @classmethod
    def instance(cls):
        if cls._instance is None:
            cls._instance = cls()
        return cls._instance

    @property
    def lang(self):
        return self._lang

    @lang.setter
    def lang(self, value):
        if value in TRANSLATIONS:
            self._lang = value

    def t(self, key, *args):
        """Get translation for key, with optional format args"""
        text = TRANSLATIONS.get(self._lang, {}).get(key, key)
        if args:
            return text.format(*args)
        return text

    def toggle_lang(self):
        if self._lang == "ru":
            self._lang = "en"
        else:
            self._lang = "ru"
        return self._lang

```

ui/main_window.py

[python] 810 lines

```
=====

# ui/main_window.py
import os
import sys
import time
import traceback
from pathlib import Path
from typing import Optional

from PyQt6.QtWidgets import (
    QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
    QPushButton, QTextEdit, QLabel, QFileDialog,
    QComboBox, QMessageBox, QTabWidget, QGroupBox,
    QStatusBar, QFrame, QApplication, QCheckBox,
    QSplitter, QSizePolicy,
)
from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer, QSize
from PyQt6.QtGui import QFont, QAction, QKeySequence, QShortcut, QPalette, QColor

from core.parser import AIFormatParser
from core.deployer import ProjectDeployer
from core.exporter import ProjectExporter
from core.project_scanner import ProjectScanner
from core.prompt_manager import PromptManager
from models.project import Project
from ui.localization import Localization

import logging

logger = logging.getLogger(__name__)

class ParseWorker(QThread):
    finished = pyqtSignal(object)
    error = pyqtSignal(str)

    def __init__(self, text):
        super().__init__()
        self.text = text

    def run(self):
        try:
            parser = AIFormatParser()
            project = parser.parse(self.text)
            self.finished.emit(project)
        except Exception as e:
            self.error.emit(str(e) + "\n\n" + traceback.format_exc())

class DeployWorker(QThread):
    finished = pyqtSignal(str)
    error = pyqtSignal(str)

    def __init__(self, project, target_dir, overwrite):
        super().__init__()
        self.project = project
        self.target_dir = target_dir
        self.overwrite = overwrite

    def run(self):
        try:
            deployer = ProjectDeployer()
            path = deployer.deploy(self.project, self.target_dir, self.overwrite)
            self.finished.emit(path)
        except Exception as e:
            self.error.emit(str(e))

class ExportPDFWorker(QThread):
    finished = pyqtSignal(str)
    error = pyqtSignal(str)

    def __init__(self, project, output_path):
```

```

        super().__init__()
        self.project = project
        self.output_path = output_path

    def run(self):
        try:
            exporter = ProjectExporter()
            exporter.export_pdf(self.project, self.output_path)
            self.finished.emit(self.output_path)
        except Exception as e:
            self.error.emit(str(e))

DARK_STYLE = """
QMainWindow { background-color: #1e1e2e; }
QWidget { background-color: #1e1e2e; color: #cdd6f4; }
QGroupBox {
    font-weight: bold; font-size: 13px;
    border: 1px solid #45475a; border-radius: 8px;
    margin-top: 12px; padding: 15px 10px 10px 10px;
    background-color: #313244; color: #cdd6f4;
}
QGroupBox::title {
    subcontrol-origin: margin; left: 15px;
    padding: 0 8px; color: #cdd6f4;
}
QTabWidget::pane {
    border: 1px solid #45475a; border-radius: 6px;
    background: #313244;
}
QTabBar::tab {
    padding: 10px 24px; font-size: 13px; font-weight: 600;
    border: 1px solid transparent; border-bottom: none;
    border-top-left-radius: 6px; border-top-right-radius: 6px;
    margin-right: 2px; background: #45475a; color: #a6adc8;
}
QTabBar::tab:selected { background: #313244; color: #89b4fa; border-color: #45475a; }
QTabBar::tab:hover:!selected { background: #585b70; color: #cdd6f4; }
QTextEdit {
    border: 2px solid #45475a; border-radius: 8px; padding: 10px;
    background-color: #1e1e2e; color: #cdd6f4;
    font-family: 'Consolas', 'Fira Code', 'Courier New', monospace; font-size: 11px;
    selection-background-color: #585b70;
}
QTextEdit:focus { border-color: #89b4fa; }
QTextEdit[readOnly="true"] {
    background-color: #11111b; color: #a6e3a1;
    border: none; border-radius: 8px; padding: 12px;
}
QComboBox {
    border: 1px solid #45475a; border-radius: 6px;
    padding: 6px 12px; background: #313244; color: #cdd6f4;
    min-height: 28px; font-size: 12px;
}
QComboBox:focus { border-color: #89b4fa; }
QComboBox::drop-down { border: none; width: 30px; }
QComboBox QAbstractItemView {
    background-color: #313244; color: #cdd6f4;
    selection-background-color: #45475a; border: 1px solid #585b70;
}
QCheckBox { color: #a6adc8; font-size: 12px; }
QCheckBox::indicator { width: 16px; height: 16px; }
QLabel { color: #cdd6f4; }
QStatusBar { background: #11111b; border-top: 1px solid #45475a; }
QStatusBar QLabel { color: #a6adc8; font-size: 12px; padding: 2px 8px; }
QMessageBox { background-color: #313244; color: #cdd6f4; }
QScrollBar:vertical {
    background: #1e1e2e; width: 10px; border: none;
}
QScrollBar::handle:vertical {
    background: #45475a; border-radius: 5px; min-height: 20px;
}
QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical { height: 0; }
"""

LIGHT_STYLE = """
QMainWindow { background-color: #f0f2f5; }
QWidget { background-color: #f0f2f5; color: #1e293b; }
QGroupBox {

```

```

    font-weight: bold; font-size: 13px;
    border: 1px solid #d1d5db; border-radius: 8px;
    margin-top: 12px; padding: 15px 10px 10px 10px;
    background-color: white; color: #1e293b;
}
QGroupBox::title {
    subcontrol-origin: margin; left: 15px;
    padding: 0 8px; color: #374151;
}
QTabWidget::pane {
    border: 1px solid #d1d5db; border-radius: 6px; background: white;
}
QTabBar::tab {
    padding: 10px 24px; font-size: 13px; font-weight: 600;
    border: 1px solid transparent; border-bottom: none;
    border-top-left-radius: 6px; border-top-right-radius: 6px;
    margin-right: 2px; background: #e5e7eb; color: #6b7280;
}
QTabBar::tab:selected { background: white; color: #1d4ed8; border-color: #d1d5db; }
QTabBar::tab:hover:!selected { background: #f3f4f6; color: #374151; }
QTextEdit {
    border: 2px solid #d1d5db; border-radius: 8px; padding: 10px;
    background-color: #fafbfc; color: #1e293b;
    font-family: 'Consolas', 'Fira Code', 'Courier New', monospace; font-size: 11px;
    selection-background-color: #bfdbfe;
}
QTextEdit:focus { border-color: #3b82f6; background-color: white; }
QTextEdit[readOnly="true"] {
    background-color: #1e293b; color: #e2e8f0;
    border: none; border-radius: 8px; padding: 12px;
}
QComboBox {
    border: 1px solid #d1d5db; border-radius: 6px;
    padding: 6px 12px; background: white; color: #1e293b;
    min-height: 28px; font-size: 12px;
}
QComboBox:focus { border-color: #3b82f6; }
QComboBox::drop-down { border: none; width: 30px; }
QComboBox QAbstractItemView {
    background-color: white; color: #1e293b;
    selection-background-color: #e0e7ff; border: 1px solid #d1d5db;
}
QCheckBox { color: #64748b; font-size: 12px; }
QLabel { color: #1e293b; }
QStatusBar { background: #f8fafc; border-top: 1px solid #e2e8f0; }
QStatusBar QLabel { color: #64748b; font-size: 12px; padding: 2px 8px; }
QScrollBar:vertical {
    background: #f0f2f5; width: 10px; border: none;
}
QScrollBar::handle:vertical {
    background: #cbd5e1; border-radius: 5px; min-height: 20px;
}
QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical { height: 0; }
"""

```

```

def _btn_style(bg, fg="white"):
    return (
        "QPushButton { background-color: " + bg + "; color: " + fg + ";"
        " border: none; padding: 10px 22px; border-radius: 6px;"
        " font-size: 13px; font-weight: 600; min-width: 100px; }"
        "QPushButton:hover { background-color: " + bg + "cc; }"
        "QPushButton:pressed { background-color: " + bg + "aa; }"
        "QPushButton:disabled { background-color: #888888; color: #bbbbbb; }"
    )

```

```

def _small_btn(bg):
    return (
        "QPushButton { background-color: " + bg + "; color: white;"
        " border: none; padding: 6px 14px; border-radius: 5px;"
        " font-size: 12px; font-weight: 600; }"
        "QPushButton:hover { background-color: " + bg + "dd; }"
    )

```

```

def _toggle_btn(bg):
    return (
        "QPushButton { background-color: " + bg + "; color: white;"

```



```

" border: none; padding: 6px 16px; border-radius: 5px;"
" font-size: 11px; font-weight: 600; min-width: 120px; }"
"QPushButton:hover { background-color: " + bg + "dd; }"
)

```

```

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.loc = Localization.instance()
        self.parser = AIFormatParser()
        self.scanner = ProjectScanner()
        self.exporter = ProjectExporter()
        self.current_project = None
        self.export_project = None
        self._workers = []
        self._dark_mode = False
        self._build_ui()
        self._build_menu()
        self._build_shortcuts()
        self._detect_directories()
        self._apply_theme()

    def t(self, key, *args):
        return self.loc.t(key, *args)

    def _apply_theme(self):
        if self._dark_mode:
            self.setStyleSheet(DARK_STYLE)
        else:
            self.setStyleSheet(LIGHT_STYLE)

    def _toggle_theme(self):
        self._dark_mode = not self._dark_mode
        self._apply_theme()
        if self._dark_mode:
            self.theme_btn.setText(self.t("theme_dark"))
        else:
            self.theme_btn.setText(self.t("theme_light"))

    def _toggle_language(self):
        self.loc.toggle_lang()
        self._refresh_all_texts()

    def _refresh_all_texts(self):
        """Update all UI texts after language change"""
        self.setWindowTitle(self.t("app_title"))

        # Directory group
        self.dir_group.setTitle(self.t("working_directory"))
        self.btn_browse_dir.setText(self.t("browse"))
        self.btn_refresh_dir.setText(self.t("refresh"))

        # Tabs
        self.tabs.setTabText(0, self.t("tab_import"))
        self.tabs.setTabText(1, self.t("tab_export"))
        self.tabs.setTabText(2, self.t("tab_help"))

        # Import tab
        self.prompt_label.setText(self.t("copy_prompt_for_ai"))
        self.input_header_label.setText(self.t("paste_ai_response"))
        self.input_text.setPlaceholderText(self.t("placeholder_input"))
        self.btn_parse.setText(self.t("analyze"))
        self.btn_deploy.setText(self.t("create_project"))
        self.overwrite_check.setText(self.t("overwrite_existing"))
        self.btn_clear.setText(self.t("clear"))
        self.result_label.setText(self.t("analysis_result"))

        # Prompt buttons
        prompt_keys = ["full", "short", "continue", "refactor", "add_feature"]
        prompt_t_keys = ["prompt_full", "prompt_short", "prompt_continue", "prompt_refactor", "prompt_add_feature"]
        for i, btn in enumerate(self.prompt_buttons):
            if i < len(prompt_t_keys):
                btn.setText(self.t(prompt_t_keys[i]))

        # Export tab
        self.sel_group.setTitle(self.t("select_project_to_export"))
        self.btn_scan.setText(self.t("find_projects"))
        self.btn_browse_proj.setText(self.t("browse_folder"))

```

```

self.btn_pdf.setText(self.t("export_pdf"))
self.btn_copy_ai.setText(self.t("copy_ai_format"))
self.export_preview_label.setText(self.t("export_preview"))

# Help tab
self.help_text.setHtml(self.t("help_content"))

# Theme & language buttons
if self._dark_mode:
    self.theme_btn.setText(self.t("theme_dark"))
else:
    self.theme_btn.setText(self.t("theme_light"))
self.lang_btn.setText(self.t("lang_label"))

# Update char count
self._on_input_changed()

# Status
self._set_status(self.t("ready"))

def _build_ui(self):
    self.setWindowTitle(self.t("app_title"))
    self.setMinimumSize(950, 700)
    self.resize(1200, 850)

    central = QWidget()
    self.setCentralWidget(central)
    root = QVBoxLayout(central)
    root.setContentsMargins(12, 8, 12, 4)
    root.setSpacing(8)

    # top bar: directory + theme/lang buttons
    top_bar = QHBoxLayout()
    top_bar.setSpacing(8)

    # directory panel
    self.dir_group = QGroupBox(self.t("working_directory"))
    dir_layout = QVBoxLayout(self.dir_group)
    dir_layout.setContentsMargins(10, 5, 10, 5)

    self.dir_combo = QComboBox()
    self.dir_combo.setEditable(True)
    self.dir_combo.setMinimumWidth(400)
    dir_layout.addWidget(self.dir_combo, stretch=1)

    self.btn_browse_dir = QPushButton(self.t("browse"))
    self.btn_browse_dir.setStyleSheet(_small_btn("#6366f1"))
    self.btn_browse_dir.clicked.connect(self._on_browse_directory)
    dir_layout.addWidget(self.btn_browse_dir)

    self.btn_refresh_dir = QPushButton(self.t("refresh"))
    self.btn_refresh_dir.setStyleSheet(_small_btn("#64748b"))
    self.btn_refresh_dir.clicked.connect(self._detect_directories)
    dir_layout.addWidget(self.btn_refresh_dir)

    top_bar.addWidget(self.dir_group, stretch=1)

    # theme & language buttons (vertical stack)
    toggle_frame = QFrame()
    toggle_layout = QVBoxLayout(toggle_frame)
    toggle_layout.setContentsMargins(0, 12, 0, 0)
    toggle_layout.setSpacing(4)

    self.theme_btn = QPushButton(self.t("theme_light"))
    self.theme_btn.setStyleSheet(_toggle_btn("#7c3aed"))
    self.theme_btn.clicked.connect(self._toggle_theme)
    toggle_layout.addWidget(self.theme_btn)

    self.lang_btn = QPushButton(self.t("lang_label"))
    self.lang_btn.setStyleSheet(_toggle_btn("#0891b2"))
    self.lang_btn.clicked.connect(self._toggle_language)
    toggle_layout.addWidget(self.lang_btn)

    top_bar.addWidget(toggle_frame)

    root.addLayout(top_bar)

    # tabs
    self.tabs = QTabWidget()

```

```

self.tabs.addTab(self._build_import_tab(), self.t("tab_import"))
self.tabs.addTab(self._build_export_tab(), self.t("tab_export"))
self.tabs.addTab(self._build_help_tab(), self.t("tab_help"))
root.addWidget(self.tabs)

# status bar
self.status_bar = QStatusBar()
self.setStatusBar(self.status_bar)
self._set_status(self.t("ready"))

def _build_import_tab(self):
    tab = QWidget()
    lay = QVBoxLayout(tab)
    lay.setContentsMargins(10, 10, 10, 10)
    lay.setSpacing(8)

    # prompt panel
    prompt_frame = QFrame()
    prompt_frame.setStyleSheet(
        "QFrame { background-color: #166534; border-radius: 8px; padding: 6px; }"
        "QLabel { color: white; }"
    )
    prompt_lay = QHBoxLayout(prompt_frame)
    prompt_lay.setContentsMargins(10, 6, 10, 6)

    self.prompt_label = QLabel(self.t("copy_prompt_for_ai"))
    prompt_lay.addWidget(self.prompt_label)

    variants = PromptManager.get_all_variants()
    prompt_keys = list(variants.keys())
    prompt_t_keys = ["prompt_full", "prompt_short", "prompt_continue", "prompt_refactor", "prompt_add_feature"]

    self.prompt_buttons = []
    for i, (key, variant) in enumerate(variants.items()):
        t_key = prompt_t_keys[i] if i < len(prompt_t_keys) else variant.title
        btn = QPushButton(self.t(t_key) if i < len(prompt_t_keys) else variant.title)
        btn.setToolTip(variant.description)
        btn.setStyleSheet(_small_btn("#

... (truncated)

```

AI Format (for reimport)

Copy text below into AI Code Deployer to recreate this project:

```
PROJSTART
METASTART
PROJECT_NAME: PythonProject
VERSION: 1.0.0
LANGUAGE: Python
INSTALL_COMMAND: pip install -r requirements.txt
METAEND
```

```
STRUCTSTART
PythonProject/
..gitignore
..AICodeDeployer.spec
..README.md
..core/
....DUNDERinitDUNDER.py
....deployer.py
....exporter.py
....parser.py
....projectUSCOREScanner.py
....promptUSCOREmanager.py
..main.py
..models/
....DUNDERinitDUNDER.py
....project.py
..requirements.txt
..ui/
....DUNDERinitDUNDER.py
....localization.py
....mainUSCOREwindow.py
STRUCTEND
```

```
FILESTART
FILEPATH: gitignore
FILETYPE: text
CODESTART
HASHSIGN Python
DUNDERpycacheDUNDER/
STARSIGN.py[cod]
STARSIGN$py.class
STARSIGN.so
STARSIGN.egg-info/
dist/
build/
STARSIGN.egg
```

```
HASHSIGN Virtual environment
venv/
.venv/
env/
.env/
```

```
HASHSIGN PyInstaller
STARSIGN.spec
!build.spec
```

```
HASHSIGN IDE
.idea/
.vscode/
STARSIGN.swp
STARSIGN.swo
```

```
HASHSIGN OS
.DSUSCOREStore
Thumbs.db
desktop.ini
```

```
HASHSIGN Logs
STARSIGN.log
```

```
HASHSIGN Distribution
output/
STARSIGN.exe
CODEEND
FILEEND
```

```

FILESTART
FILEPATH: AICodeDeployer.spec
FILETYPE: text
CODESTART
HASHSIGN -STARSIGN- mode: python ; coding: utf-8 -STARSIGN-
from PyInstaller.utils.hooks import collectUSCOREall

datas = []
binaries = []
hiddenimports = ['PIL', 'PIL.Image', 'PIL.ImageDraw', 'PIL.ImageFont', 'PIL.ImageTk', 'PyQt6', 'PyQt6.QtCore', 'PyQt6.QtGui', 'PyQt6.QtWidgets']
tmpUSCOREet = collectUSCOREall('reportlab')
datas += tmpUSCOREet[0]; binaries += tmpUSCOREet[1]; hiddenimports += tmpUSCOREet[2]
tmpUSCOREet = collectUSCOREall('PIL')
datas += tmpUSCOREet[0]; binaries += tmpUSCOREet[1]; hiddenimports += tmpUSCOREet[2]

a = Analysis(
    ....['main.py'],
    ....pathex=['C:/Users/yanix/PycharmProjects/PythonProject'],
    ....binaries=binaries,
    ....datas=datas,
    ....hiddenimports=hiddenimports,
    ....hookspath=[],
    ....hooksconfig={},
    ....runtimeUSCOREhooks=[],
    ....excludes=['IPython', 'boto3', 'botocore', 'cv2', 'django', 'fastapi', 'flask', 'jupyter', 'jupyterlab', 'keras', 'matplotlib', 'notebook'],
    ....noarchive=False,
    ....optimize=0,
)
pyz = PYZ(a.pure)

exe = EXE(
    ....pyz,
    ....a.scripts,
    ....a.binaries,
    ....a.datas,
    ....[],
    ....name='AICodeDeployer',
    ....debug=False,
    ....bootloaderUSCOREignoreUSCOREsignals=False,
    ....strip=False,
    ....upx=True,
    ....upxUSCOREexclude=[],
    ....runtimeUSCOREetmpdir=None,
    ....console=False,
    ....disableUSCOREwindowedUSCOREetraceback=False,
    ....argvUSCOREemulation=False,
    ....targetUSCOREearch=None,
    ....codesignUSCOREidentity=None,
    ....entitlementsUSCOREfile=None,
)

CODEEND
FILEEND

FILESTART
FILEPATH: core/DUNDERinitDUNDER.py
FILETYPE: python
CODESTART
HASHSIGN core/DUNDERinitDUNDER.py
from .parser import AIFormatParser
from .deployer import ProjectDeployer
from .exporter import ProjectExporter
from .projectUSCOREscanner import ProjectScanner
from .promptUSCOREmanager import PromptManager
CODEEND
FILEEND

FILESTART
FILEPATH: core/deployer.py
FILETYPE: python
CODESTART
HASHSIGN core/deployer.py
import os
import shutil
from pathlib import Path
from typing import Optional
from models.project import Project
import logging

```

```

logger = logging.getLogger(DUNDERnameDUNDER)

class ProjectDeployer:
    .... """Создание проекта на диске из модели Project"""

    ....def deploy(
        .....self,
        .....project: Project,
        .....targetUSCoredir: str,
        .....overwrite: bool = False
        ....) -> str:
        ..... """
        .....Развернуть проект.

        .....Args:
        .....project: Объект проекта
        .....targetUSCoredir: Корневая директория для проектов
        .....overwrite: Перезаписывать ли существующие файлы

        .....Returns:
        .....Путь к созданному проекту
        ..... """
        .....projectUSCoredir = Path(targetUSCoredir) / project.metadata.name

        .....HASHSIGN Проверка существования
        .....if projectUSCoredir.exists():
        .....if not overwrite:
        .....HASHSIGN Добавляем суффикс
        .....counter = 1
        .....while projectUSCoredir.exists():
        .....projectUSCoredir = Path(targetUSCoredir) / f"{project.metadata.name}USCORE{counter}"
        .....counter += 1
        .....else:
        .....logger.warning(f"Перезаписываю: {projectUSCoredir}")

        .....logger.info(f"Создаю проект: {projectUSCoredir}")

        .....HASHSIGN Создаём директорию проекта
        .....projectUSCoredir.mkdir(parents=True, existUSCOREok=True)

        .....HASHSIGN Создаём файлы
        .....createdUSCOREfiles = 0
        .....for pf in project.files:
        .....fileUSCOREpath = projectUSCoredir / pf.relativeUSCOREpath

        .....HASHSIGN Создаём поддиректории
        .....fileUSCOREpath.parent.mkdir(parents=True, existUSCOREok=True)

        .....HASHSIGN Записываем файл
        .....try:
        .....fileUSCOREpath.writeUSCOREtext(pf.content, encoding='utf-8')
        .....createdUSCOREfiles += 1
        .....logger.debug(f" Создан: {pf.relativeUSCOREpath}")
        .....except Exception as e:
        .....logger.error(f" Ошибка создания {pf.relativeUSCOREpath}: {e}")

        .....HASHSIGN Создаём README если его нет
        .....readmeUSCOREexists = any(
        .....f.relativeUSCOREpath.lower() in ('readme.md', 'readme.txt', 'readme')
        .....for f in project.files
        ..... )
        .....if not readmeUSCOREexists:
        .....self.USCOREcreateUSCOREreadme(project, projectUSCoredir)

        .....logger.info(
        .....f"Проект создан: {createdUSCOREfiles}/{len(project.files)} файлов "
        .....f"в {projectUSCoredir}"
        ..... )

        .....return str(projectUSCoredir)

    ....def USCOREcreateUSCOREreadme(self, project: Project, projectUSCoredir: Path):
    ..... """Автоматически создать README.md"""
    .....meta = project.metadata
    .....lines = [f"# {meta.name}"]

    .....if meta.description:
    .....lines.extend(["", meta.description])

```

```

.....lines.extend(["", "## Структура", "BACKTICKBACKTICKBACKTICK"])
.....lines.append(project.getUSCOREtree())
.....lines.append("BACKTICKBACKTICKBACKTICK")

.....if meta.installUSCOREcommand:
.....lines.extend(["", "## Установка", f"BACKTICKBACKTICKBACKTICKbash\n{meta.installUSCOREcommand}\nBACKTICKBACKTICKBACKTICK"])

.....if meta.runUSCOREcommand:
.....lines.extend(["", "## Заныск", f"BACKTICKBACKTICKBACKTICKbash\n{meta.runUSCOREcommand}\nBACKTICKBACKTICKBACKTICK"])

.....if meta.buildUSCOREcommand:
.....lines.extend(["", "## Сборка", f"BACKTICKBACKTICKBACKTICKbash\n{meta.buildUSCOREcommand}\nBACKTICKBACKTICKBACKTICK"])

.....lines.extend([
....."", "---",
.....f"STARSIGNСгенерировано AI Code DeployerSTARSIGN"
.....])

.....readmeUSCOREpath = projectUSCOREdir / "README.md"
.....readmeUSCOREpath.writeUSCOREtext("\n".join(lines), encoding='utf-8')
CODEEND
FILEEND

FILESTART
FILEPATH: core/exporter.py
FILETYPE: python
CODESTART
HASHSIGN core/exporter.py
import os
import sys
from pathlib import Path
from typing import Optional
from models.project import Project, ProjectFile, ProjectMetadata
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import mm, cm
from reportlab.platypus import (
....SimpleDocTemplate, Paragraph, Spacer, Preformatted,
....Table, TableStyle, PageBreak
)
from reportlab.lib.enums import TAUSCORELEFT, TAUSCORECENTER
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
import logging

logger = logging.getLogger(DUNDERnameDUNDER)

def USCOREfindUSCOREsystemUSCOREfont():
....candidates = []
....if sys.platform == "win32":
.....windir = os.environ.get("WINDIR", "C:\\Windows")
.....fontsUSCOREdir = os.path.join(windir, "Fonts")
.....candidates = [
.....os.path.join(fontsUSCOREdir, "consola.ttf"),
.....os.path.join(fontsUSCOREdir, "cour.ttf"),
.....os.path.join(fontsUSCOREdir, "lucon.ttf"),
.....os.path.join(fontsUSCOREdir, "arial.ttf"),
.....os.path.join(fontsUSCOREdir, "segoeui.ttf"),
.....]
....elif sys.platform == "darwin":
.....candidates = [
....."/System/Library/Fonts/Menlo.ttc",
....."/System/Library/Fonts/Courier.dfont",
....."/Library/Fonts/Arial.ttf",
.....]
....else:
.....candidates = [
....."/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
....."/usr/share/fonts/truetype/liberation/LiberationMono-Regular.ttf",
....."/usr/share/fonts/truetype/ubuntu/UbuntuMono-R.ttf",
....."/usr/share/fonts/TTF/DejaVuSansMono.ttf",
.....]
....for path in candidates:
.....if os.path.exists(path):
.....return path
....return None

```

```

def USCOREfindUSCOREsystemUSCOREfontUSCOREbold():
    ....if sys.platform == "win32":
    .....windir = os.environ.get("WINDIR", "C:\\Windows")
    .....fontsUSCOREDir = os.path.join(windir, "Fonts")
    .....candidates = [
    .....os.path.join(fontsUSCOREDir, "consolab.ttf"),
    .....os.path.join(fontsUSCOREDir, "courbd.ttf"),
    .....os.path.join(fontsUSCOREDir, "arialbd.ttf"),
    .....os.path.join(fontsUSCOREDir, "segoeui.ttf"),
    .....]
    ....elif sys.platform == "darwin":
    .....candidates = [
    ....."/System/Library/Fonts/Menlo.ttc",
    ....."/Library/Fonts/Arial Bold.ttf",
    .....]
    ....else:
    .....candidates = [
    ....."/usr/share/fonts/truetype/dejavu/DejaVuSansMono-Bold.ttf",
    ....."/usr/share/fonts/truetype/liberation/LiberationMono-Bold.ttf",
    ....."/usr/share/fonts/truetype/ubuntu/UbuntuMono-B.ttf",
    .....]
    ....for path in candidates:
    .....if os.path.exists(path):
    .....return path
    ....return None

def USCOREfindUSCOREsansUSCOREfont():
    ....if sys.platform == "win32":
    .....windir = os.environ.get("WINDIR", "C:\\Windows")
    .....fontsUSCOREDir = os.path.join(windir, "Fonts")
    .....candidates = [
    .....os.path.join(fontsUSCOREDir, "segoeui.ttf"),
    .....os.path.join(fontsUSCOREDir, "arial.ttf"),
    .....os.path.join(fontsUSCOREDir, "tahoma.ttf"),
    .....]
    ....elif sys.platform == "darwin":
    .....candidates = [
    ....."/System/Library/Fonts/Helvetica.ttc",
    ....."/Library/Fonts/Arial.ttf",
    .....]
    ....else:
    .....candidates = [
    ....."/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    ....."/usr/share/fonts/truetype/liberation/LiberationSans-Regular.ttf",
    ....."/usr/share/fonts/truetype/ubuntu/Ubuntu-R.ttf",
    .....]
    ....for path in candidates:
    .....if os.path.exists(path):
    .....return path
    ....return None

def USCOREfindUSCOREsansUSCOREfontUSCOREbold():
    ....if sys.platform == "win32":
    .....windir = os.environ.get("WINDIR", "C:\\Windows")
    .....fontsUSCOREDir = os.path.join(windir, "Fonts")
    .....candidates = [
    .....os.path.join(fontsUSCOREDir, "segoeui.ttf"),
    .....os.path.join(fontsUSCOREDir, "arialbd.ttf"),
    .....os.path.join(fontsUSCOREDir, "tahomabd.ttf"),
    .....]
    ....elif sys.platform == "darwin":
    .....candidates = [
    ....."/Library/Fonts/Arial Bold.ttf",
    .....]
    ....else:
    .....candidates = [
    ....."/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
    ....."/usr/share/fonts/truetype/liberation/LiberationSans-Bold.ttf",
    ....."/usr/share/fonts/truetype/ubuntu/Ubuntu-B.ttf",
    .....]
    ....for path in candidates:
    .....if os.path.exists(path):
    .....return path
    ....return None

```

```

class ProjectExporter:

```



```

...MONOUSCOREFONT = "Courier"
...MONOUSCOREFONTUSCOREBOLD = "Courier-Bold"
...SANSUSCOREFONT = "Helvetica"
...SANSUSCOREFONTUSCOREBOLD = "Helvetica-Bold"

...def DUNDERinitDUNDER(self):
.....self.USCOREregisterUSCOREfonts()
.....self.styles = self.USCOREcreateUSCOREstyles()

...def USCOREregisterUSCOREfonts(self):
.....mono = USCOREfindUSCOREsystemUSCOREfont()
.....if mono:
.....try:
.....pdfmetrics.registerFont(TTFont("MonoCyr", mono))
.....self.MONOUSCOREFONT = "MonoCyr"
.....logger.info("Mono font registered: " + mono)
.....except Exception as e:
.....logger.warning("Failed to register mono font: " + str(e))

.....monoUSCOREbold = USCOREfindUSCOREsystemUSCOREfontUSCOREbold()
.....if monoUSCOREbold:
.....try:
.....pdfmetrics.registerFont(TTFont("MonoCyrBold", monoUSCOREbold))
.....self.MONOUSCOREFONTUSCOREBOLD = "MonoCyrBold"
.....except Exception:
.....self.MONOUSCOREFONTUSCOREBOLD = self.MONOUSCOREFONT

.....sans = USCOREfindUSCOREsansUSCOREfont()
.....if sans:
.....try:
.....pdfmetrics.registerFont(TTFont("SansCyr", sans))
.....self.SANSUSCOREFONT = "SansCyr"
.....logger.info("Sans font registered: " + sans)
.....except Exception as e:
.....logger.warning("Failed to register sans font: " + str(e))

.....sansUSCOREbold = USCOREfindUSCOREsansUSCOREfontUSCOREbold()
.....if sansUSCOREbold:
.....try:
.....pdfmetrics.registerFont(TTFont("SansCyrBold", sansUSCOREbold))
.....self.SANSUSCOREFONTUSCOREBOLD = "SansCyrBold"
.....except Exception:
.....self.SANSUSCOREFONTUSCOREBOLD = self.SANSUSCOREFONT

...def USCOREcreateUSCOREstyles(self):
.....styles = getSampleStyleSheet()

.....HASHSIGN override built-in styles to use our fonts
.....styles["Title"].fontName = self.SANSUSCOREFONTUSCOREBOLD
.....styles["Title"].fontSize = 20
.....styles["Title"].leading = 26

.....styles["Heading1"].fontName = self.SANSUSCOREFONTUSCOREBOLD
.....styles["Heading1"].fontSize = 14
.....styles["Heading1"].leading = 18

.....styles["Normal"].fontName = self.SANSUSCOREFONT
.....styles["Normal"].fontSize = 10
.....styles["Normal"].leading = 13

.....styles.add(ParagraphStyle(
.....name='CodeBlock',
.....fontName=self.MONOUSCOREFONT,
.....fontSize=7,
.....leading=9,
.....leftIndent=10,
.....rightIndent=10,
.....backColor=colors.Color(0.95, 0.95, 0.95),
.....borderColor=colors.Color(0.8, 0.8, 0.8),
.....borderWidth=0.5,
.....borderPadding=5,
.....spaceBefore=5,
.....spaceAfter=10,
.....))

.....styles.add(ParagraphStyle(
.....name='FilePath',
.....fontName=self.MONOUSCOREFONTUSCOREBOLD,
.....fontSize=10,

```

```

.....leading=14,
.....textColor=colors.Color(0.1, 0.3, 0.6),
.....spaceBefore=15,
.....spaceAfter=5,
.....))

.....styles.add(ParagraphStyle(
.....name='MetaKey',
.....fontName=self.SANSUSCOREFONTUSCOREBOLD,
.....fontSize=9,
.....textColor=colors.Color(0.3, 0.3, 0.3),
.....))

.....styles.add(ParagraphStyle(
.....name='MetaValue',
.....fontName=self.SANSUSCOREFONT,
.....fontSize=9,
.....))

.....styles.add(ParagraphStyle(
.....name='TreeStyle',
.....fontName=self.MONOUSCOREFONT,
.....fontSize=8,
.....leading=11,
.....leftIndent=15,
.....backColor=colors.Color(0.97, 0.97, 1.0),
.....borderPadding=8,
.....spaceBefore=5,
.....spaceAfter=10,
.....))

.....return styles

....def exportUSCOREpdf(self, project, outputUSCOREpath):
.....doc = SimpleDocTemplate(
.....outputUSCOREpath,
.....pagesize=A4,
.....rightMargin=1.5 STARSIGN cm,
.....leftMargin=1.5 STARSIGN cm,
.....topMargin=2 STARSIGN cm,
.....bottomMargin=2 STARSIGN cm,
.....title="Project: " + project.metadata.name,
.....author="AI Code Deployer"
.....)

.....story = []

.....HASHSIGN title page
.....story.append(Spacer(1, 4 STARSIGN cm))
.....story.append(Paragraph(
.....self.USCOREescape(project.metadata.name),
.....self.styles['Title']
.....))

.....if project.metadata.description:
.....story.append(Spacer(1, 0.5 STARSIGN cm))
.....story.append(Paragraph(
.....self.USCOREescape(project.metadata.description),
.....self.styles['Normal']
.....))

.....story.append(Spacer(1, 1 STARSIGN cm))

.....metaUSCOREdata = self.USCOREbuildUSCOREmetaUSCOREtable(project.metadata)
.....if metaUSCOREdata:
.....table = Table(metaUSCOREdata, colwidths=[4 STARSIGN cm, 12 STARSIGN cm])
.....table.setStyle(TableStyle([
.....('FONTNAME', (0, 0), (0, -1), self.SANSUSCOREFONTUSCOREBOLD),
.....('FONTNAME', (1, 0), (1, -1), self.SANSUSCOREFONT),
.....('FONTSIZE', (0, 0), (-1, -1), 9),
.....('TEXTCOLOR', (0, 0), (0, -1), colors.Color(0.3, 0.3, 0.3)),
.....('VALIGN', (0, 0), (-1, -1), 'TOP'),
.....('BOTTOPPADDING', (0, 0), (-1, -1), 4),
.....('TOPPADDING', (0, 0), (-1, -1), 4),
.....]))
.....story.append(table)

.....story.append(Spacer(1, 0.5 STARSIGN cm))
.....statsUSCOREtext = "Files: " + str(project.fileUSCOREcount) + " | Lines: " + str(project.totalUSCORElines)

```

```

.....story.append(Paragraph(statsUSCOREtext, self.styles['Normal']))

.....story.append(PageBreak())

.....HASHSIGN structure
.....story.append(Paragraph("Project Structure", self.styles['Heading1']))
.....story.append(Preformatted(project.getUSCOREtree(), self.styles['TreeStyle']))
.....story.append(Spacer(1, 0.5 STARSIGN cm))
.....story.append(PageBreak())

.....HASHSIGN files
.....story.append(Paragraph("Project Files", self.styles['Heading1']))
.....story.append(Spacer(1, 0.3 STARSIGN cm))

.....for i, pf in enumerate(project.files):
.....story.append(Paragraph("=" STARSIGN 60, self.styles['Normal']))
.....story.append(Paragraph(
.....self.USCOREescape(pf.relativeUSCOREpath),
.....self.styles['FilePath']
.....))

.....if pf.description:
.....story.append(Paragraph(
....."<i>" + self.USCOREescape(pf.description) + "</i>",
.....self.styles['Normal']
.....))

.....langUSCOREinfo = "[" + pf.language + "]" if pf.language else ""
.....linesUSCOREcount = len(pf.content.splitlines())
.....infoUSCOREtext = langUSCOREinfo + " " + str(linesUSCOREcount) + " lines"
.....story.append(Paragraph(
....."<font size=7 color='gray'>" + infoUSCOREtext + "</font>",
.....self.styles['Normal']
.....))

.....codeUSCOREtext = pf.content
.....if len(codeUSCOREtext) > 15000:
.....codeUSCOREtext = codeUSCOREtext[:15000] + "\n\n... (truncated)"

.....story.append(Preformatted(codeUSCOREtext, self.styles['CodeBlock']))
.....story.append(Spacer(1, 0.5 STARSIGN cm))

.....if (i + 1) % 3 == 0 and i < len(project.files) - 1:
.....story.append(PageBreak())

.....HASHSIGN ai format section
.....story.append(PageBreak())
.....story.append(Paragraph("AI Format (for reimport)", self.styles['Heading1']))
.....story.append(Paragraph(
....."Copy text below into AI Code Deployer to recreate this project:",
.....self.styles['Normal']
.....))
.....story.append(Spacer(1, 0.3 STARSIGN cm))

.....aiUSCOREtext = project.toUSCOREaiUSCOREformat()
.....if len(aiUSCOREtext) > 30000:
.....aiUSCOREtext = aiUSCOREtext[:30000] + "\n\n... (truncated)"

.....story.append(Preformatted(aiUSCOREtext, self.styles['CodeBlock']))

.....doc.build(story)
.....logger.info("PDF created: " + outputUSCOREpath)
.....return outputUSCOREpath

...def exportUSCOREtext(self, project):
.....return project.toUSCOREaiUSCOREformat()

...def USCOREbuildUSCOREmetaUSCOREtable(self, meta):
.....rows = []
.....fields = [
.....("Version:", meta.version),
.....("Author:", meta.author),
.....("Language:", meta.language),
.....("Framework:", meta.framework),
.....("Run:", meta.runUSCOREcommand),
.....("Build:", meta.buildUSCOREcommand),
.....("Install:", meta.installUSCOREcommand),
.....]
.....for label, value in fields:

```

```

.....if value:
.....rows.append([label, value])
.....return rows

....def USCOREescape(self, text):
.....return (text
.....replace('&', '&amp;')
.....replace('<', '&lt;')
.....replace('>', '&gt;')
.....)
CODEEND
FILEEND

FILESTART
FILEPATH: core/parser.py
FILETYPE: python
CODESTART
HASHSIGN core/parser.py
import re
from typing import Optional, List
from pathlib import Path
from models.project import Project, ProjectFile, ProjectMetadata
from core.promptUSCOREmanager import restoreUSCOREcode
import logging

logger = logging.getLogger(DUNDERnameDUNDER)

class AIFormatParser:

....def parse(self, text: str) -> Project:
.....text = text.strip()
.....if "PROJSTART" in text:
.....return self.USCOREparseUSCOREplain(text)
.....if "===PROJECTUSCORESTART===" in text:
.....return self.USCOREparseUSCORElegacyUSCOREstrict(text)
.....if "[PROJECTUSCOREBEGIN]" in text:
.....return self.USCOREparseUSCORElegacyUSCOREbracket(text)
.....return self.USCOREparseUSCOREmarkdown(text)

....def USCOREparseUSCOREplain(self, text: str) -> Project:
.....project = Project()
.....start = text.find("PROJSTART")
.....end = text.find("PROJEND")
.....if start != -1:
.....text = text[start + 9:]
.....if end != -1:
.....text = text[:text.find("PROJEND")]
.....metaUSCOREstart = text.find("METASTART")
.....metaUSCOREend = text.find("METAEND")
.....if metaUSCOREstart != -1 and metaUSCOREend != -1:
.....metaUSCOREtext = text[metaUSCOREstart + 9:metaUSCOREend]
.....project.metadata = self.USCOREparseUSCOREmeta(metaUSCOREtext)
.....structUSCOREstart = text.find("STRUCTSTART")
.....structUSCOREend = text.find("STRUCTEND")
.....if structUSCOREstart != -1 and structUSCOREend != -1:
.....rawUSCOREstruct = text[structUSCOREstart + 11:structUSCOREend].strip()
.....project.structureUSCOREdescription = restoreUSCOREcode(rawUSCOREstruct)
.....fileUSCOREchunks = text.split("FILESTART")
.....for chunk in fileUSCOREchunks[1:]:
.....endUSCOREepos = chunk.find("FILEEND")
.....if endUSCOREepos != -1:
.....chunk = chunk[:endUSCOREepos]
.....pf = self.USCOREparseUSCOREplainUSCOREfile(chunk)
.....if pf:
.....project.files.append(pf)
.....logger.info("Parsed: " + project.metadata.name + " " + str(len(project.files)) + " files")
.....return project

....def USCOREparseUSCOREmeta(self, text: str) -> ProjectMetadata:
.....meta = ProjectMetadata()
.....fieldUSCOREmap = {
....."PROJECTUSCORENAME": "name",
....."DESCRIPTION": "description",
....."VERSION": "version",
....."AUTHOR": "author",
....."LANGUAGE": "language",
....."FRAMEWORK": "framework",
....."PYTHONUSCOREVERSION": "pythonUSCOREversion",

```

```

....."NODEUSCOREVERSION": "nodeUSCOREversion",
....."RUNUSCORECOMMAND": "runUSCOREcommand",
....."BUILDUSCORECOMMAND": "buildUSCOREcommand",
....."INSTALLUSCORECOMMAND": "installUSCOREcommand",
.....}
.....restored = restoreUSCOREcode(text)
.....for line in restored.strip().splitlines():
.....line = line.strip()
.....if not line:
.....continue
.....colon = line.find(":")
.....if colon == -1:
.....continue
.....key = line[colon].strip().upper()
.....value = line[colon + 1:].strip()
.....if key in fieldUSCOREmap:
.....setattr(meta, fieldUSCOREmap[key], value)
.....else:
.....meta.extra[key] = value
.....return meta

....def USCOREparseUSCOREplainUSCOREfile(self, chunk: str) -> Optional[ProjectFile]:
.....lines = chunk.strip().splitlines()
.....path = ""
.....filetype = ""
.....description = ""
.....codeUSCORElines = []
.....inUSCOREcode = False
.....for line in lines:
.....stripped = line.strip()
.....if stripped == "CODESTART":
.....inUSCOREcode = True
.....continue
.....if stripped == "CODEEND":
.....inUSCOREcode = False
.....continue
.....if inUSCOREcode:
.....codeUSCORElines.append(line)
.....continue
.....if stripped.startswith("FILEPATH:"):
.....path = stripped[9:].strip()
.....elif stripped.startswith("FILETYPE:"):
.....filetype = stripped[9:].strip()
.....elif stripped.startswith("FILEDESC:"):
.....description = stripped[9:].strip()
.....if not path:
.....return None
.....HASHSIGN восстанавливаем плейсхолдеры в пути
.....path = restoreUSCOREcode(path)
.....HASHSIGN если путь gitignore без точки то добавляем точку
.....if path.endswith("gitignore") and not path.endswith(".gitignore"):
.....parts = path.rsplit("gitignore", 1)
.....path = parts[0] + ".gitignore"
.....description = restoreUSCOREcode(description)
.....filetype = restoreUSCOREcode(filetype)
.....rawUSCOREcode = "\n".join(codeUSCORElines)
.....restoredUSCOREcode = restoreUSCOREcode(rawUSCOREcode)
.....pf = ProjectFile(relativeUSCOREpath=path, content=restoredUSCOREcode, language=filetype, description=description)
.....if not pf.language:
.....pf.autoUSCOREdetectUSCORElanguage()
.....return pf

....def USCOREparseUSCORElegacyUSCOREstrict(self, text: str) -> Project:
.....project = Project()
.....match = re.search(r'===PROJECTUSCORESTART===(.STARSIGN?)===PROJECTUSCOREEND===', text, re.DOTALL)
.....if match:
.....text = match.group(1)
.....metaUSCOREmatch = re.search(r'---METAUSCORESTART---(.STARSIGN?)---METAUSCOREEND---', text, re.DOTALL)
.....if metaUSCOREmatch:
.....project.metadata = self.USCOREparseUSCOREmeta(metaUSCOREmatch.group(1))
.....structUSCOREmatch = re.search(r'---STRUCTUREUSCORESTART---(.STARSIGN?)---STRUCTUREUSCOREEND---', text, re.DOTALL)
.....if structUSCOREmatch:
.....project.structureUSCOREdescription = structUSCOREmatch.group(1).strip()
.....fileUSCOREblocks = re.findall(r'---FILEUSCORESTART---(.STARSIGN?)---FILEUSCOREEND---', text, re.DOTALL)
.....for block in fileUSCOREblocks:
.....pf = self.USCOREparseUSCORElegacyUSCOREfile(block)
.....if pf:
.....project.files.append(pf)
.....return project

```

```

....def USCOREparseUSCORELegacyUSCOREbracket(self, text: str) -> Project:
.....project = Project()
.....match = re.search(r'\[PROJECTUSCOREBEGIN\](.STARSIGN?)\[PROJECTUSCOREFINISH\]', text, re.DOTALL)
.....if match:
.....    text = match.group(1)
.....    metaUSCOREmatch = re.search(r'\[METAUSCOREBEGIN\](.STARSIGN?)\[METAUSCOREFINISH\]', text, re.DOTALL)
.....    if metaUSCOREmatch:
.....        project.metadata = self.USCOREparseUSCOREmeta(metaUSCOREmatch.group(1))
.....    structUSCOREmatch = re.search(r'\[STRUCTUSCOREBEGIN\](.STARSIGN?)\[STRUCTUSCOREFINISH\]', text, re.DOTALL)
.....    if structUSCOREmatch:
.....        project.structureUSCOREdescription = structUSCOREmatch.group(1).strip()
.....        fileUSCOREblocks = re.findall(r'\[FILEUSCOREBEGIN\](.STARSIGN?)\[FILEUSCOREFINISH\]', text, re.DOTALL)
.....        for block in fileUSCOREblocks:
.....            pf = self.USCOREparseUSCORELegacyUSCOREfile(block)
.....            if pf:
.....                project.files.append(pf)
.....    return project

....def USCOREparseUSCORELegacyUSCOREfile(self, block: str) -> Optional[ProjectFile]:
.....lines = block.strip().splitlines()
.....path = ""
.....description = ""
.....contentUSCORElines = []
.....inUSCOREcode = False
.....for line in lines:
.....    stripped = line.strip()
.....    if not inUSCOREcode:
.....        if stripped.startswith("PATH:"):
.....            path = stripped[5:].strip()
.....        elif stripped.startswith("FILEPATH:"):
.....            path = stripped[9:].strip()
.....        elif stripped.startswith("DESCRIPTION:"):
.....            description = stripped[12:].strip()
.....        elif stripped.startswith("FILEDESC:"):
.....            description = stripped[9:].strip()
.....        elif stripped.startswith("BACKTICKBACKTICKBACKTICK") or stripped == "CODESTART" or stripped == "[CODEUSCOREBEGIN]":
.....            inUSCOREcode = True
.....        else:
.....            if stripped.startswith("BACKTICKBACKTICKBACKTICK") or stripped == "CODEEND" or stripped == "[CODEUSCOREFINISH]":
.....                inUSCOREcode = False
.....            else:
.....                contentUSCORElines.append(line)
.....    if not path:
.....        return None
.....    pf = ProjectFile(relativeUSCOREpath=path, content="\n".join(contentUSCORElines), description=description)
.....    pf.autoUSCOREdetectUSCORElanguage()
.....    return pf

....def USCOREparseUSCOREMarkdown(self, text: str) -> Project:
.....project = Project()
.....titleUSCOREmatch = re.search(r'^#\s+(.)$', text, re.MULTILINE)
.....if titleUSCOREmatch:
.....    name = titleUSCOREmatch.group(1).strip()
.....    name = re.sub(r'[STARSIGNUSCOREBACKTICK]', '', name)
.....    project.metadata.name = self.USCOREslugify(name)
.....    pattern = re.compile(
.....        r'(?:(?:\n)(?:#{1,6}\s+[BACKTICKSTARSIGN]STARSIGN(?:^\nBACKTICKSTARSIGN)+\.\w+)[BACKTICKSTARSIGN]STARSIGN|[BACKTICKSTARSIGN]{1,3})'
.....    )
... (truncated)

```